14장
데이터와 함수를 한 묶음으로 묶어 재사용하기: 클래스와 객체

이 장에서는 클래스와 객체를 배워 보겠습니다. 클래스는 자주 사용하는 변수를 묶어서 표현하는 점에서 사용자 정의 함수와 유사한 면이 있습니다. 다만, 사용자 정의 함수는 기능을 재사용하는 부분에 중점을 맞추었고 클래스는 함수에 데이터까지 포함해 재사용하는 개념입니다. 설명만으로는 이해하기 어려운 개념이니 실습하며 자세히 살펴봅시다.

14.1

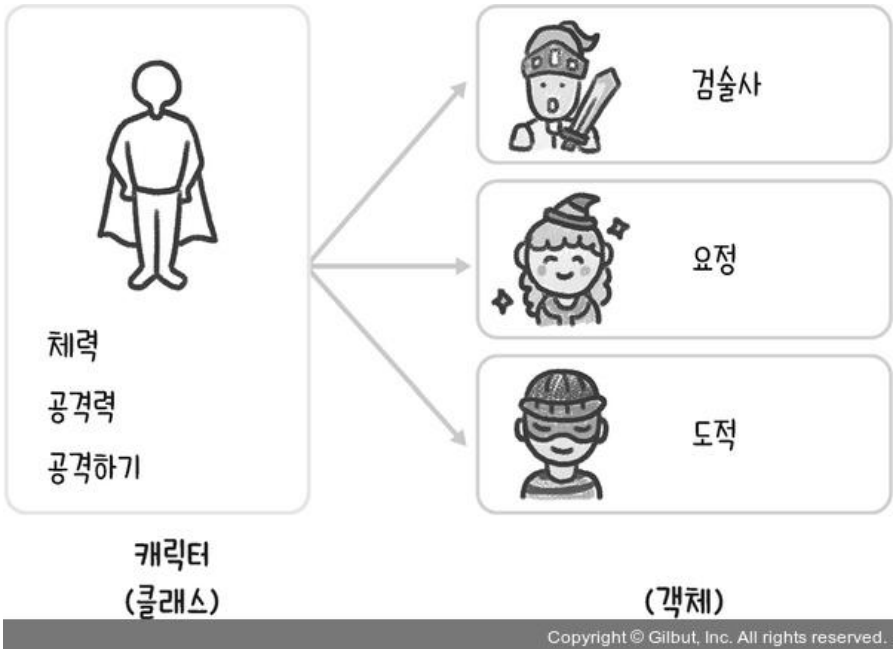
자주 사용되는 단위를 클래스로 정의하기

14.1.1 클래스와 객체

클래스와 객체의 관계

게임을 예로 들어 봅시다. 게임을 처음 시작하면 캐릭터 선택하기 화면이 나옵니다. 여기서 원하는 캐릭터를 고를 수 있는데, 캐릭터들은 공통 특성을 가지고 있습니다. 예를 들면 체력이나 공격력, 방어력 등의 능력치죠. 그리고 공통 기능도 가지고 있습니다. 공격하기, 방어하기, 전진하기, 후퇴하기 등의 기술이죠. 게임을 설계할 때 게임에 등장하는 캐릭터들이 공통으로 가져야 하는 특성과 기능을 정해 둔 것입니다. 이처럼 캐릭터들이 가져야 할 공통 특성과 기능을 모아 놓은 것이 클래스(class)입니다. 즉, 모든 캐릭터가 가져야 하는 형식을 정의한 것이죠.

여기에서는 간단하게, 모든 캐릭터가 체력, 공격력이라는 특성과 공격하기 기능이 있다고 가정하고 캐릭터 클래스(캐릭터의 형식)를 만들어 봅시다. 그리고 캐릭터의 특성과 기능을 바탕으로 검술사, 요정, 도적이라는 실제 캐릭터를 만든다고 합시다. 형식을 바탕으로 만들어진 모든 캐릭터는 체력, 공격력과 공격하기라는 공통 특성과 기능을 가집니다.



▲ 그림 14-1 클래스와 객체의 관계

게임에서 '캐릭터'가 클래스라면 검술사, 요정, 도적은 '캐릭터' 클래스를 바탕으로 구현된 실체입니다. 이처럼 클래스를 바탕으로 만들어진 실체를 객체(object)라고 합니다.

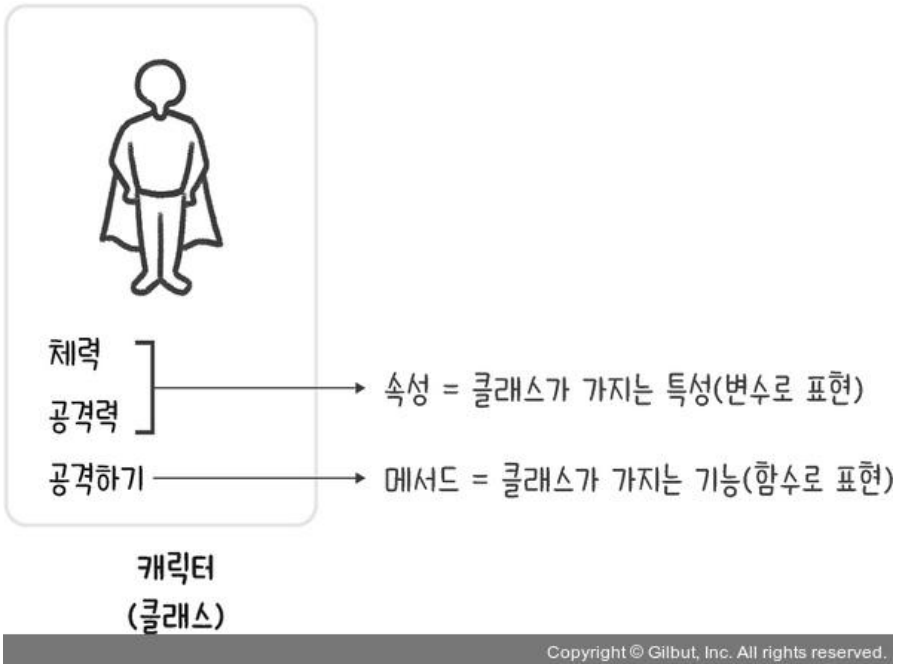
정리하면 클래스는 형식이고, 객체는 형식을 바탕으로 만들어지는 실체를 뜻합니다. 형식이 없는데 실체를 만들 순 없습니다. 따라서 개별 값을 가진 실체(객체)를 만들기 전에 형식(클래스)을 먼저 정의해야 합니다.

TIP 객체를 인스턴스(instance)라고도 표현합니다. 인스턴스는 클래스와 객체의 관계를 중심으로 설명할 때 많이 씁니다. 어떤 클래스에 속한 객체를 표현할 때 인스턴스라고 하죠. 처음 배울 때 두 용어를 모두 사용하면 헷갈릴 수 있으니 여기서는 객체라는 표현으로 통일해서 사용하겠습니다.

속성과 메서드

객체인 검술사, 요정, 도적은 캐릭터 클래스의 특성인 체력과 공격력을 공통으로 가집니다. 객체마다 특성의 구체적인 값은 다릅니다. 예를 들어, 검술사는 체력이 100이지만, 요정은 체력이 60일 수 있습니다. 공격력도 객체마다 값이 다 다릅니다. 값이 달라지므로 이를 변수에 저장합니다. 클래스의 특성을 저장하는 변수를 속성(attribute)이라고 합니다. 속성의 사전적 의미가 '사물의 특징이나 성질'이므로 클래스의 속성이란 '클래스가 가진 특징'을 의미합니다.

캐릭터 클래스에는 공격하기라는 기능도 있습니다. 캐릭터 클래스에 공격하기 기능이 있다면, 이 클래스를 바탕으로 생성한 검술사, 요정, 도적 객체도 동일한 기능을 갖게 됩니다. 클래스의 기능은 앞서 배운 사용자 정의 함수로 표현할 수 있습니다. 클래스의 기능을 표현하는 사용자 정의 함수를 메서드(method)라고 합니다. 메서드는 사용자 정의 함수와 딱 한 가지만 다르고 다른 부분은 모두 동일한데, 이 부분은 차차 배워 가겠습니다. 속성과 메서드는 모두 클래스에 포함됩니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 14-2 속성과 메서드

캐릭터 클래스에서 모든 캐릭터는 체력과 공격력이라는 속성을 가져야 한다고 정해 놨습니다. 클래스를 바탕으로 만든 검술사, 요정, 도적이라는 객체는 속성에 구체적이고 개별적인 값을 가지게 됩니다. 예를 들어, 검술사의 체력은 100, 공격력은 10입니다. 요정은 각각 60, 5이고 도적은 85, 8이죠. 검술사, 요정, 도적 모두 체력과 공격력이라는 특성을 가지고 있지만 그 값은 각각 다릅니다.

이처럼 동일한 속성이나 기능을 가지고 있지만 세부 값만 변경해 여러 번 사용해야 할 때, 클래스로 미리 정의할 수 있습니다. 그리고 필요할 때마다 클래스에 개별 값을 넣어 객체를 만들고요.

14.1.2 클래스 정의하고 객체 만들기

클래스 정의와 객체 생성 형식

클래스가 먼저 만들어져야 이를 바탕으로 객체를 만들 수 있습니다. 실체를 만들기 전에 형식을 미리 정의해야 하는 것이죠. 따라서 프로그램은 클래스 정의 → 객체 생성의 순서로 작성해야 합니다. 먼저 클래스를 정의해 봅시다.

클래스는 앞에서 설명했듯이 속성과 메서드로 구성됩니다. 이를 다음 형식으로 정의합니다.

형식

```
class 클래스명:
    def __init__(self, a, b):
        속성1 = a
        속성2 = b
    def 메서드1():
    def 메서드2():
```

사용자 정의 함수를 만들 때 `def` 키워드에 함수명과 콜론(:)을 붙여 작성했죠. 클래스를 만들 때도 `class` 키워드에 클래스명과 콜론(:)을 붙여 작성합니다. 클래스명은 보통 관습적으로 CapWords 규칙에 따라 단어의 첫 글자를 대문자로 적습니다. 그리고 클래스명 뒤에는 소괄호(`()`)를 넣지 않습니다.

클래스 내부를 살펴봅시다. 가장 먼저 `__init__()`을 정의합니다. 객체가 생성될 때 가장 처음으로 실행되는 부분으로, 뒤에서 자세히 다루겠습니다. `__init__()` 아래에는 클래스에 포함될 메서드를 작성합니다.

클래스를 정의하고 나면 객체를 다음과 같이 만듭니다. 클래스보다 굉장히 간단하죠?

형식

```
객체명 = 클래스명()
```

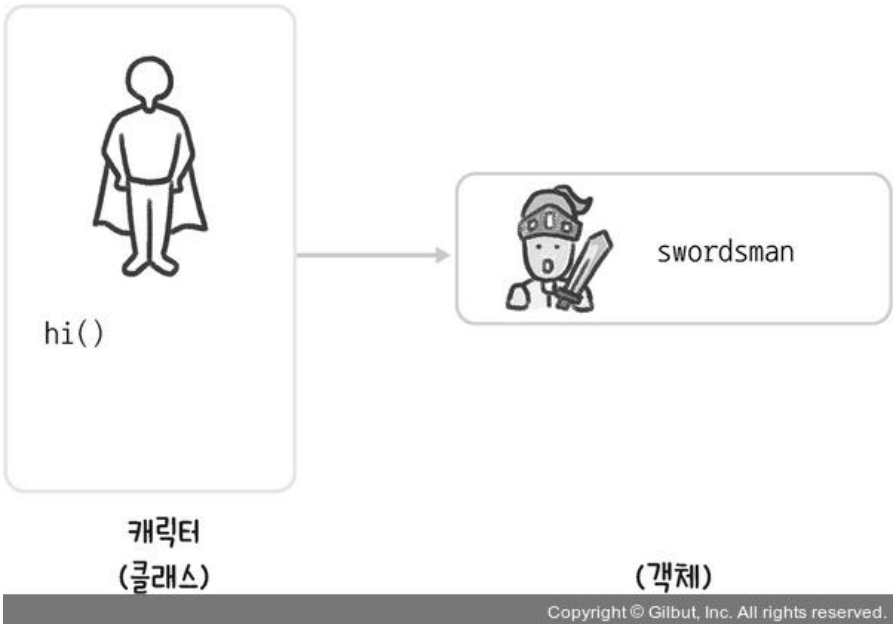
그럼 클래스부터 만들어 봅시다. 게임 캐릭터의 형식을 담은 클래스를 만들어 보겠습니다. 게임 캐릭터는 여러 속성과 메서드를 갖고 있지만, 학습을 위해 작은 것부터 완성해 나가겠습니다. 클래스명은 `Character`고, 메서드 `hi()`를 가진 클래스를 정의해 봅시다.

```
class Character:
    def hi():
        print('안녕하세요')
```

클래스를 정의했으니 객체를 만들어야죠. 객체를 만드는 방법에 따라 검술사를 의미하는 `swordsman`이라는 객체를 만들어 봅시다.

```
swordsman = Character()
```

클래스명인 `Character`에 소괄호(`()`)를 붙이면 `Character` 클래스를 바탕으로 하는 객체가 생성됩니다. 이 객체를 `swordsman`이라는 이름으로 저장합니다. 이제 `Character` 클래스를 형식으로 하는 `swordsman` 객체가 만들졌습니다.



▲ 그림 14-3 Character 클래스 정의와 swordsman 객체 생성

`swordsman`의 타입을 확인해 봅시다.

```
print(type(swordsman))
```

실행결과
```
<class '__main__.Character'>
```

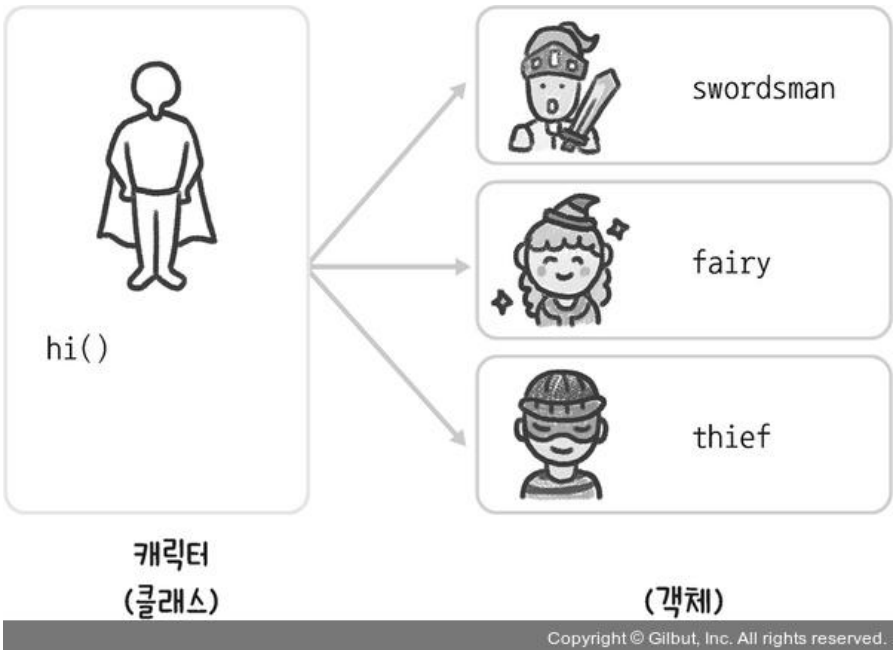
`__main__`의 `Character` 클래스라고 나옵니다. `swordsman`은 `Character` 클래스의 형식을 가지고 있다는 의미입니다.

> NOTE __main__
>
> 실행결과에 나온 `__main__`은 실행파일을 뜻합니다. 파이썬에서는 다양한 라이브러리를 추가해 사용할 수 있습니다. 앞에서 `random`이나 `matplotlib` 같은 라이브러리를 추가해서 사용했죠. `random`, `matplotlib`은 별도의 파이썬 파일로 존재하는데, 이 파일을 우리가 만든 파이썬 파일에 `import`해서 함께 실행합니다. 이때 우리가 만든 실행파일이 `__main__`으로 설정되고, `import`된 라이브러리는 `__main__`이 아닌 파일로 설정됩니다. 따라서 실행결과의 `__main__.Character`는 실행파일에 정의된 `Character` 클래스라는 의미입니다.

요정 객체와 도적 객체도 만들어 봅시다.

```
fairy = Character()
thief = Character()
```

이제 `Character` 클래스를 바탕으로 3개의 객체가 만들어졌습니다.



▲ 그림 14-4 Character 클래스와 생성된 객체들

메서드 정의하고 사용하기

메서드는 클래스 내부에 정의된 사용자 정의 함수입니다. 함수는 미리 정의된 기능을 수행하죠. 13장의 `plus(a, b)`는 두 수를 전달받아 더하는 기능을, `gugudan()`은 구구단 문제를 출제하는 기능을 수행합니다. 메서드 역시 함수이므로 함수처럼 미리 정의하고 호출되면 해당 기능을 수행합니다.

현재 `Character` 클래스에는 인사하는 기능인 `hi()` 메서드가 있습니다. 따라서 `swordsman`, `fairy`, `thief` 객체도 `hi()` 메서드를 사용할 수 있습니다. `swordsman` 객체가 인사한다고 해 봅시다. 그럼 `swordsman`이 `hi()`를 실행해야 합니다. 객체가 메서드를 실행(호출)하는 방법은 `객체명.메서드()`입니다. 따라서 `swordsman.hi()`로 작성하면 됩니다. 한번 실행해 봅시다.

```
swordsman.hi()
```

```
실행결과
----> 1 swordsman.hi()
TypeError: hi() takes 0 positional arguments but 1 was given
```

메서드를 호출한 `swordsman.hi()`에서 오류가 발생합니다. 오류 내용에는 '`hi()`는 위치 인자가 0개인데, 1개가 주어졌다'고 나옵니다. 이게 무슨 뜻일까요?

`hi()` 메서드를 정의할 때 매개변수를 넣지 않았습니다. 전달받을 것이 없기 때문이죠. 그런데 오류 내용에서는 인자 1개를 넣어 호출했다고 나옵니다. 코드에서도 `swordsman.hi()`로 인자 없이 호출했는데 말이죠. 이 부분이 바로 사용자 정의 함수와 메서드의 다른 점입니다.

메서드에는 유의해야 할 점이 있습니다. 메서드를 `객체명.메서드()`로 호출할 때 메서드 앞에 붙은 객체명이 자동으로 메서드의 인자로 전달됩니다. 즉, `swordsman.hi()`를 하면 자동으로 `swordsman` 객체가 `hi()`의 인자로 넘어간다는 뜻입니다. 따라서 클래스의 메서드를 정의할 때 자동으로 넘어오는 인자를 받을 매개변수가 필요합니다. 이럴 때 파이썬에서는 `self`를 사용합니다. `self`는 메서드를 호출한 객체를 의미하는 아주 중요한 키워드입니다.

```
class Character:
    def hi(self):
        print('안녕하세요.')
swordsman = Character()
swordsman.hi()
```

```
실행결과
안녕하세요.
```

메서드에 매개변수로 `self`를 추가하니 오류가 발생하지 않습니다. `self`가 자동으로 넘어오는 인자를 받기 때문입니다. self는 '자기 자신'이라는 뜻으로, 여기서는 메서드를 호출한 `swordsman` 객체 자신을 가리킵니다.

TIP 자동으로 전달되는 객체를 받는 매개변수로 `self` 대신 다른 이름을 사용해도 오류가 발생하지 않습니다. 하지만 파이썬에서는 자기 자신의 객체를 전달받는다는 의미로 `self`를 관습적으로 사용합니다.

`self`는 클래스와 객체에서 반드시 이해해야 하는 개념입니다.



▲ 그림 14-5 메서드의 매개변수 self

그럼 다음 코드에서 `self`로 전달되는 인자는 무엇일까요?

```
fairy = Character()
fairy.hi()
```

`self`로 전달되는 인자는 `fairy` 객체입니다.

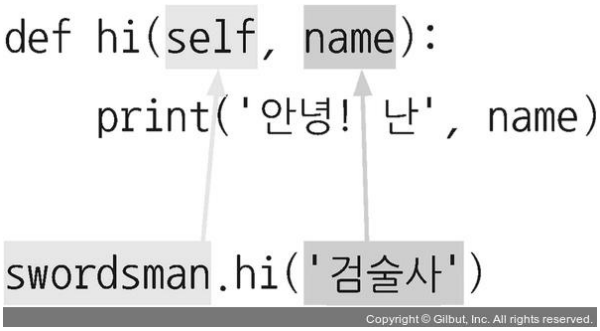
메서드는 클래스 내부에 있는 사용자 정의 함수입니다. 따라서 매개변수를 `self` 외에도 더 추가할 수 있습니다. `hi()` 메서드에 매개변수 `name`으로 이름을 전달받아 출력해 봅시다.

```
class Character:
    def hi(self, name):
        print('안녕! 난', name)

swordsman = Character()
swordsman.hi('검술사')
fairy = Character()
fairy.hi('요정')
thief = Character()
thief.hi('도적')
```

```
실행결과
안녕! 난 검술사
안녕! 난 요정
안녕! 난 도적
```

`hi()` 메서드에서 `self` 뒤에 매개변수 `name`을 추가했습니다. 매개변수가 여러 개일 때는 항상 `self`를 가장 먼저 적어야 합니다. 메서드가 호출될 때 객체를 가장 먼저 전달하기 때문입니다. 매개변수 `name`을 추가했으므로 `hi()`를 호출할 때 `name`에 해당하는 인자를 넣어야 합니다. 인자로 각 캐릭터 이름을 넣고 메서드를 호출하니 오류 없이 출력됩니다. 단, `name`은 메서드 내부에서만 사용 가능한 지역변수이므로 `hi()` 메서드 외부에서는 사용할 수 없습니다.



▲ 그림 14-6 메서드에 매개변수가 여럿일 때

이번에는 공격하기 기능인 `attack()` 메서드를 추가로 작성해 봅시다. 공격할 대상을 매개변수 `target`으로 전달받아 '000을/를 공격합니다!'를 출력해 보겠습니다.

```
class Character:
    def hi(self, name):
        print('안녕! 난', name)
    def attack(self, target):
        print(target, '을/를 공격합니다.')

swordsman = Character()
swordsman.hi('검술사')
swordsman.attack('도적')
swordsman.attack('요정')
```

```
실행결과
안녕! 난 검술사
도적 을/를 공격합니다.
요정 을/를 공격합니다.
```

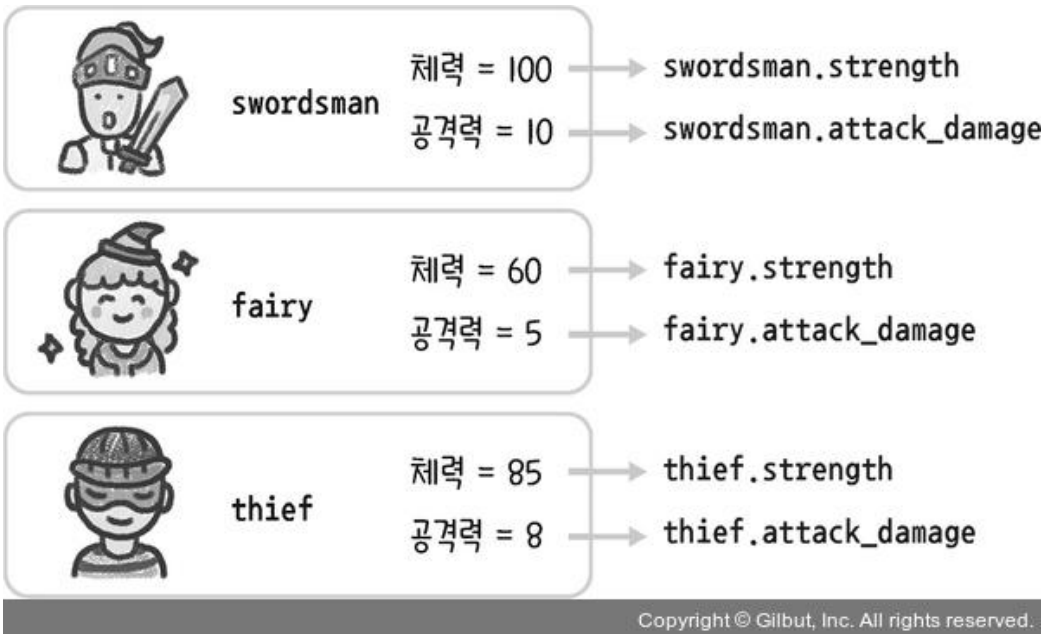
`attack()` 메서드는 `hi()` 메서드와 형식이 비슷합니다. `self`를 첫 번째 매개변수로 작성해서 자동으로 전달되는 객체를 담습니다. 그리고 추가한 매개변수 `target`으로 공격 대상의 이름을 전달받아 메서드 내부의 `print()`를 실행합니다.

생성자와 인스턴스 속성

이제 속성을 배워 봅시다. 속성은 객체가 공통으로 가지는 특성이라고 했습니다. 속성에는 클래스 속성과 인스턴스 속성이 있는데 인스턴스 속성을 많이 사용하므로 먼저 배워 보겠습니다.

검술사, 요정, 도적은 모두 체력과 공격력이라는 속성을 가지고 있습니다. 그런데 캐릭터별로 체력과 공격력이 다르죠. 검술사의 체력은 100이지만, 요정과 도적의 체력은 각각 60과 85입니다. 공격력도 객체별로 다릅니다. 이처럼 객체별로 개별적인 값을 가진 속성을 인스턴스 속성이라고 합니다.

인스턴스 속성은 `객체명.속성`으로 접근할 수 있습니다. 체력의 속성명을 `strength`로, 공격력의 속성명을 `attack_damage`로 정한다고 할 때, 검술사의 체력은 `swordsman.strength`, 요정의 공격력은 `fairy.attack_damage`로 접근할 수 있습니다.



▲ 그림 14-7 객체의 인스턴스 속성

이렇게 객체별로 속성에 개별적인 값을 저장하려면 생성자(constructor)라는 특별한 메서드를 정의해야 합니다. 생성자는 객체가 생성될 때 자동으로 실행되는 메서드로, 객체별로 속성에 다른 값을 저장할 때 유용합니다. 생성자는 이름이 이미 정해져 있습니다. 앞에서 클래스를 정의할 때 나온 `__init__()`이 바로 생성자입니다. `__init__()`을 클래스 내부에서 사용하면 해당 메서드가 생성자가 됩니다.

> TIP 앞뒤에 언더스코어(_)가 2개씩 있으니 유의하세요! 언더스코어가 없거나 개수가 맞지 않으면 오류가 발생합니다.

`Character` 클래스에 생성자를 추가해 봅시다. 생성자를 사용하면 객체 생성과 동시에 인스턴스 속성을 만들 수 있습니다. 가령 검술사라는 객체 생성과 동시에 체력은 100, 공격력은 10으로 설정하는 것이죠. 클래스는 객체가 생성될 때 필요한 속성값을 전달받아야 하므로 `_ _init__()`에 매개변수가 있어야 합니다. 이름은 매개변수 `name`으로, 체력은 `hp`로, 공격력은 `ad`로 전달받겠습니다. `hi()` 메서드를 삭제하고 그 자리에 생성자 `__init__()`을 추가해 봅시다.

```
class Character:
    def __init__(self, name, hp, ad):
        self.name = name
        self.strength = hp
        self.attack_damage = ad
        print('Character 클래스의 객체 생성 =>', self.name)
    def attack(self, target):
        print(target, '을/를 공격합니다.')
```

생성자는 객체가 생성될 때 바로 실행되므로 클래스의 가장 상단에 두는 것이 일반적입니다. `__init__()` 역시 메서드이므로 `self`를 첫 번째 매개변수로 넣어야 합니다. 그리고 캐릭터 이름, 체력, 공격력을 전달받는 매개변수 `name`, `hp`, `ad`를 차례로 넣습니다.

인스턴스 속성은 `객체명.속성`으로 접근한다고 했죠? 생성자에서 객체는 `self`로 전달되므로 `self.name`, `self.strength`, `self.attack_damage`에 매개변수로 전달받은 값을 각각 저장합니다. 그리고 객체 생성이 완료됐음을 알리는 안내문과 캐릭터 이름을 출력합니다.

객체를 다시 만들어 봅시다. 기존 방식으로 검술사를 만들어 보겠습니다.

```
swordsman = Character()
```

```
실행결과
TypeError: __init__() missing 3 required positional arguments: 'name', 'hp', and 'ad'
```

오류가 발생합니다. 오류 내용을 보면 인자 3개가 없다고 나옵니다. 생성자 `__init__()`은 객체를 생성하면 자동으로 실행됩니다. 따라서 `swordsman = Character()`로 객체를 생성할 때 생성자도 실행됩니다. 그런데 객체를 생성하면서 어떤 인자도 넣어 주지 않았습니다. 결국 `_ _init__()`에서 전달받은 인자는 객체 자신인 `self` 하나입니다.

`self`는 자동으로 전달되지만 `name`, `hp`, `ad`는 직접 값을 전달받아야 하는 매개변수입니다. 따라서 객체를 생성할 때 `Character()`의 소괄호 안에 `name`, `hp`, `ad`에 해당하는 값(인자)을 넣어줘야 합니다. 객체를 생성하는 코드에 인자를 넣고 실행해 봅시다.

```
swordsman = Character('검술사', 100, 10)
```

```
실행결과
Character 클래스의 객체 생성 => 검술사
```

이번에는 오류 없이 객체가 생성됩니다. 객체를 만들 때 자동으로 생성자 `__init__()`이 실행되고, 첫 매개변수 `self`에 객체 이름인 `swordsman`이 전달됩니다. 그럼 `__init__()` 내부에서 `self.name = name`을 실행할 때 `self`에는 `swordsman`이, 매개변수 `name`에는 '검술사'가 저장된 상태입니다. 즉 `self.name = name`은 `swordsman.name = '검술사'`로 실행되는 것이죠. 동일한 원리로 객체 `swordsman` 내에 `strength`, `attack_damage` 속성을 만들고 각각 100, 10을 저장하게 됩니다. 인스턴스 속성에 값이 잘 저장됐는지 확인해 봅시다.

```
print(swordsman.name, swordsman.strength, swordsman.attack_damage)
```

```
실행결과
검술사 100 10
```

`swordsman` 객체의 인스턴스 속성값이 출력됩니다. 그런데 객체를 생성할 때마다 속성값이 잘 설정됐는지 일일이 출력하는 것은 비효율적이겠죠? 인스턴스 속성을 알려 주는 `information()` 메서드를 추가해 봅시다.

```
class Character:
    def __init__(self, name, hp, ad):
        self.name = name
        self.strength = hp
        self.attack_damage = ad
        print('Character 클래스의 객체 생성 =>', self.name)
    def information(self):
        print('이름 =>', self.name,  ', 체력 =>', self.strength, ', 공격력 =>', self.attack_da
mage)
    def attack(self, target):
        print(target,'을/를 공격합니다.')

swordsman = Character('검술사', 100, 10)
swordsman.information()
```

```
실행결과
Character 클래스의 객체 생성 => 검술사
이름 => 검술사 , 체력 => 100 , 공격력 => 10
```

인스턴스 속성을 출력하는 `information()` 메서드를 작성했습니다. 검술사 객체를 생성하고 나서 `swordsman.information()`을 실행하니 검술사가 가진 속성이 한 번에 보입니다.

요정과 도적 객체도 생성해 봅시다.

```
fairy = Character('요정', 60, 5)
fairy.information()

thief = Character('도적', 85, 8)
thief.information()
```

```
실행결과
Character 클래스의 객체 생성 => 요정
이름 => 요정 , 체력 => 60 , 공격력 => 5
Character 클래스의 객체 생성 => 도적
이름 => 도적 , 체력 => 85 , 공격력 => 8
```

요정 객체를 생성할 때 `__init__()`에 전달되는 `self`는 `fairy`입니다. 따라서 생성자 내에서 `fairy.name`, `fairy.strength`, `fairy.attack_damage`에 각각 `'요정'`, `60`, `5`를 저장합니다. 도적 객체를 생성할 때도 `self`에 `thief`를 전달한 후 동일한 원리가 적용됩니다.

인스턴스 속성 변경하기

게임을 플레이하는 도중에 캐릭터가 레벨 업하면 체력이 20% 증가되고, 공격력도 5 증가한다고 해 봅시다. 그렇다면 이미 생성된 객체의 인스턴스 속성을 변경해야 합니다. 이처럼 프로그램 실행 중에 인스턴스 속성의 값을 변경하고 싶을 때 `객체명.속성 = 값` 형식으로 직접 값을 넣어 줄 수 있습니다. 예를 들어, 검술사의 체력을 120으로 올리려면 `swordsman.strength = 120`을 실행하면 됩니다. 공격력도 15로 다시 설정하려면 같은 방법으로 `swordsman.attack_damage = 15`를 실행하면 됩니다.

```
print('변경 전 ----------')
swordsman.information()

swordsman.strength = 120
swordsman.attack_damage = 15

print('변경 후 ----------')
swordsman.information()
```

```
실행결과
변경 전 ----------
이름 => 검술사 , 체력 => 100 , 공격력 => 10
변경 후 ----------
이름 => 검술사 , 체력 => 120 , 공격력 => 15
```

이와 같이 이미 생성한 객체의 인스턴스 속성값을 클래스 외부에서 직접 변경하고 출력할 수 있습니다.

또한, 메서드로 인스턴스 속성값을 변경할 수도 있습니다. 체력을 20% 증가시키고, 공격력은 5 증가시키는 `level_up()`이라는 메서드를 추가해 봅시다.

```
class Character:
    def __init__(self, name, hp, ad):
        ...
    def information(self):
        ...
    def attack(self, target):
        ...
    def level_up(self):
        self.strength *= 1.2
        self.attack_damage += 5

swordsman = Character('검술사', 100, 10)
print('변경 전 ----------')
swordsman.information()

swordsman.level_up()
print('변경 후 ----------')
swordsman.information()
```

```
실행결과
Character 클래스의 객체 => 검술사
변경 전 ----------
이름 => 검술사 , 체력 => 100 , 공격력 => 10
변경 후 ----------
이름 => 검술사 , 체력 => 120.0 , 공격력 => 15
```

`level_up()` 역시 메서드이므로 자동으로 전달되는 객체를 받아 줄 매개변수 `self`가 필요합니다. 그리고 메서드 내부에서 인스턴스 속성인 `self.strength`가 20% 증가하도록 1.2를 곱하고, `self.attack_damage`가 5 증가하도록 5를 더합니다. 클래스 외부에서 객체명을 붙여 `level_up()`을 실행하면 해당 객체의 인스턴스 속성이 변경됩니다.

메서드의 매개변수로 객체 전달하기
이번에는 한 캐릭터가 다른 캐릭터를 공격해 체력이 감소되는 상황을 구현해 보겠습니다. 객체가 다른 객체를 공격하는 상황입니다.

검술사가 요정을 공격하면 요정의 체력이 감소되어야 합니다. 이때 공격하는 캐릭터의 공격력만큼 상대방 캐릭터의 체력이 감소됩니다. 이를 구현하려면 `attack()` 메서드에 공격 대상인 객체를 인자로 전달해야 합니다. 공격 대상을 받는 매개변수는 `target`입니다. 그리고 메서드 내부에서 해당 캐릭터의 체력을 감소시켜야죠. `target` 자체가 하나의 객체이므로 공격받는 캐릭터의 체력(`target.strength`)을 공격하는 캐릭터의 공격력(`self.attack_damage`)만큼 감소시킵니다.

```
class Character:
    def __init__(self, name, hp, ad):
        ...
    def information(self):
        ...
    def attack(self, target):
        print(self.name,'이/가', target.name, '(', target.strength, ') 을/를 공격합니다.')
        # 공격받은 대상의 체력을 공격력만큼 감소시키기
        target.strength -= self.attack_damage
        print('공격 성공!', target.name, '(', target.strength, ')')
    def level_up(self):
    ...

# 검술사, 요정, 도적 객체 생성
swordsman = Character('검술사', 100, 10)
fairy = Character('요정', 60, 5)
thief = Character('도적', 85, 8)

# 검술사가 요정을 공격
swordsman.attack(fairy)
```

실행결과

```
Character 클래스의 객체 생성 => 검술사
Character 클래스의 객체 생성 => 요정
Character 클래스의 객체 생성 => 도적
검술사 이/가 요정 ( 60 ) 을/를 공격합니다.
공격 성공! 요정 ( 50 )
```

`target` 자체가 객체이므로 `attack()`의 인자로 객체명을 넘겨주면 됩니다. `swordsman`이 `attack()` 메서드를 호출했으므로 `self`에는 `swordsman`이 전달되고 `target`에는 `fairy`가 전달됩니다. 도적이 검술사를 공격하는 코드와 요정이 도적을 공격하는 코드도 작성해 봅시다.

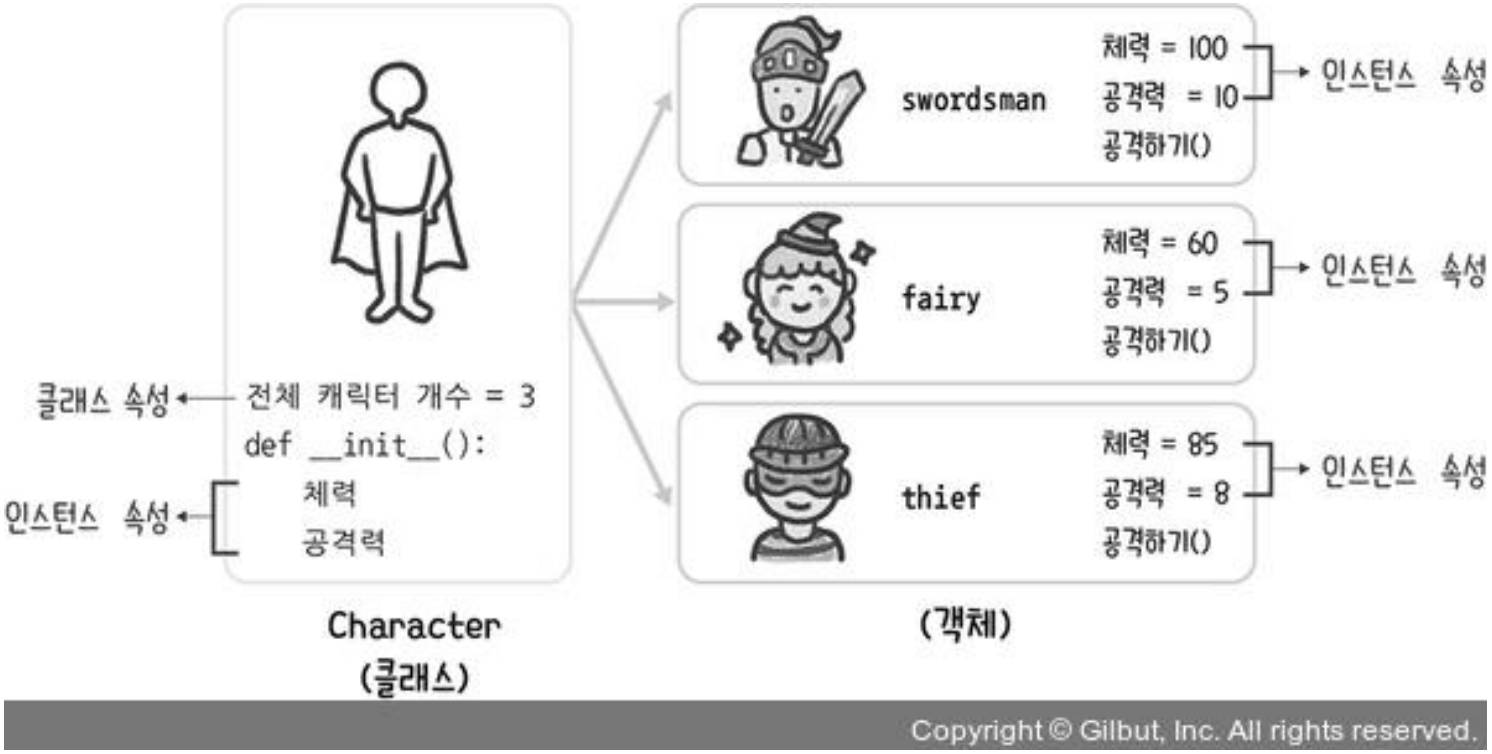
```
thief.attack(swordsman)   # 도적이 검술사를 공격
fairy.attack(thief)       # 요정이 도적을 공격
```

```
실행결과
도적 이/가 검술사 ( 100 ) 을/를 공격합니다.
공격 성공! 검술사 ( 92 )
요정 이/가 도적 ( 85 ) 을/를 공격합니다.
공격 성공! 도적 ( 80 )
```

클래스 속성

속성에는 클래스 속성과 인스턴스 속성 두 가지가 있다고 했죠? 지금까지 인스턴스 속성을 배웠으니 클래스 속성도 다뤄 보겠습니다. 클래스 속성은 클래스 내부에 만든 변수로, 클래스가 직접 값을 가지고 있는 속성을 의미합니다.

가령 게임에서 전체 캐릭터 개수를 관리해야 한다고 해 봅시다. 캐릭터 객체 하나가 생길 때마다 전체 캐릭터 개수는 1씩 증가합니다. 전체 캐릭터 개수가 인스턴스 속성이라면 한 객체에서만 접근할 수 있습니다. 이렇게 되면 새로운 객체가 생성될 때 전체 캐릭터 개수를 수정할 수 없습니다. 그래서 모든 객체가 접근할 수 있는 클래스 속성을 만듭니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 14-8 인스턴스 속성과 클래스 속성

클래스 속성은 클래스 내부의 메서드 외부 공간에 정의하면 됩니다. 인스턴스 속성에 `객체명.속성`으로 접근하듯이 메서드에서 클래스 속성에 접근할 때는 클래스명.속성으로 접근합니다. 전체 캐릭터 개수를 나타내는 속성을 `total_character_cnt`라 하고 객체가 하나 새로 생길 때마다 1을 증가시키겠습니다. 그러려면 생성자 내부에 `total_character_cnt`를 증가시키는 명령어가 있어야 합니다. 다음 코드를 실행해 봅시다.

```
class Character:
    # 클래스 속성
    total_character_cnt = 0

    def __init__(self, name, hp, ad):
        self.name = name
        self.strength = hp
        self.attack_damage = ad
        print('Character 클래스의 객체 =>', self.name)
        # 객체 생성 시 클래스 속성 1씩 증가시키기
        Character.total_character_cnt += 1
        print('현재 캐릭터 개수 =', Character.total_character_cnt)
    def information(self):
        ...
    def attack(self, target):
        ...
    def level_up(self):
        ...

# 검술사, 요정, 도적 객체 생성
swordsman = Character('검술사', 100, 10)
fairy = Character('요정', 60, 5)
thief = Character('도적', 85, 8)
```

```
실행결과
Character 클래스의 객체 => 검술사
현재 캐릭터 개수 = 1
Character 클래스의 객체 => 요정
현재 캐릭터 개수 = 2
Character 클래스의 객체 => 도적
현재 캐릭터 개수 = 3
```

클래스에 클래스 속성을 만들었습니다. 클래스 속성도 변수이므로 초깃값 0을 넣어 줍니다. 이렇게 만든 클래스 속성은 해당 클래스를 바탕으로 만든 모든 객체에서 접근할 수 있습니다. 생성자는 객체가 생성될 때 같이 실행되므로 `total_character_cnt`를 1씩 증가시키는 코드를 생성자 내부에 추가합니다. 메서드에서 클래스 속성을 다루므로 클래스명을 붙여 `Character.total_character_cnt`로 사용해야 오류가 발생하지 않습니다. 캐릭터를 새로 만들 때마다 현재 캐릭터 개수가 하나씩 증가하는 것이 보입니다.

정리하면, 클래스 속성은 클래스 내부에 존재하며 모든 객체에서 접근할 수 있고 모든 객체에 영향을 미칩니다.

14.1.3 클래스와 사용자 정의 함수 비교

클래스는 앞서 배운 사용자 정의 함수처럼 같은 기능을 여러 번 사용하려고 할 때 필요합니다. 사용자 정의 함수는 자주 사용하는 기능을 미리 정의하고, 기능이 필요할 때마다 정의해 둔 함수명으로 간단하게 호출합니다. 클래스 역시 자주 사용하는 단위를 미리 만들어 놓고, 필요할 때마다 사용하는 방식입니다. 다른 점은 사용자 정의 함수에서는 자주 사용하는 단위가 하나의 기능이었습니다. 예를 들어, 숫자의 합을 구하는 기능, 구구단 문제를 내는 기능 등이죠. 하지만 클래스의 단위는 하나 이상의 기능과 변수까지 포함합니다. 다시 말해 하나 이상의 사용자 정의 함수(메서드)와 변수(속성)를 합쳐 단위를 만들고 이를 클래스로 정의합니다.

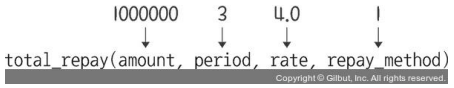
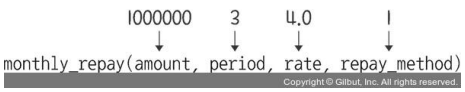
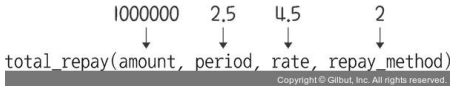
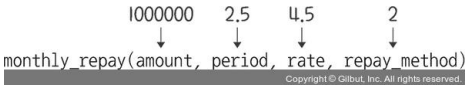
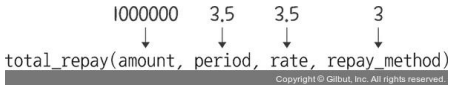
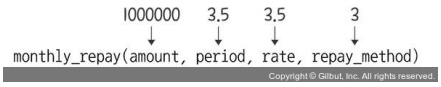
학자금을 대출받는 상황을 가정해 봅시다. 대출 상품에 따라 상환 방식도 다르고, 금리도 다릅니다. 비교하고 싶은 조건은 다음과 같습니다.

▼ 표 14-1 학자금 대출 조건

구분	대출 금액	대출 기간	대출 금리	상환 방식
조건 1	100만 원	3년	4.0%	원리금 균등 상환
조건 2	100만 원	2.5년	4.5%	원금 균등 상환
조건 3	100만 원	3.5년	3.5%	만기 일시 상환

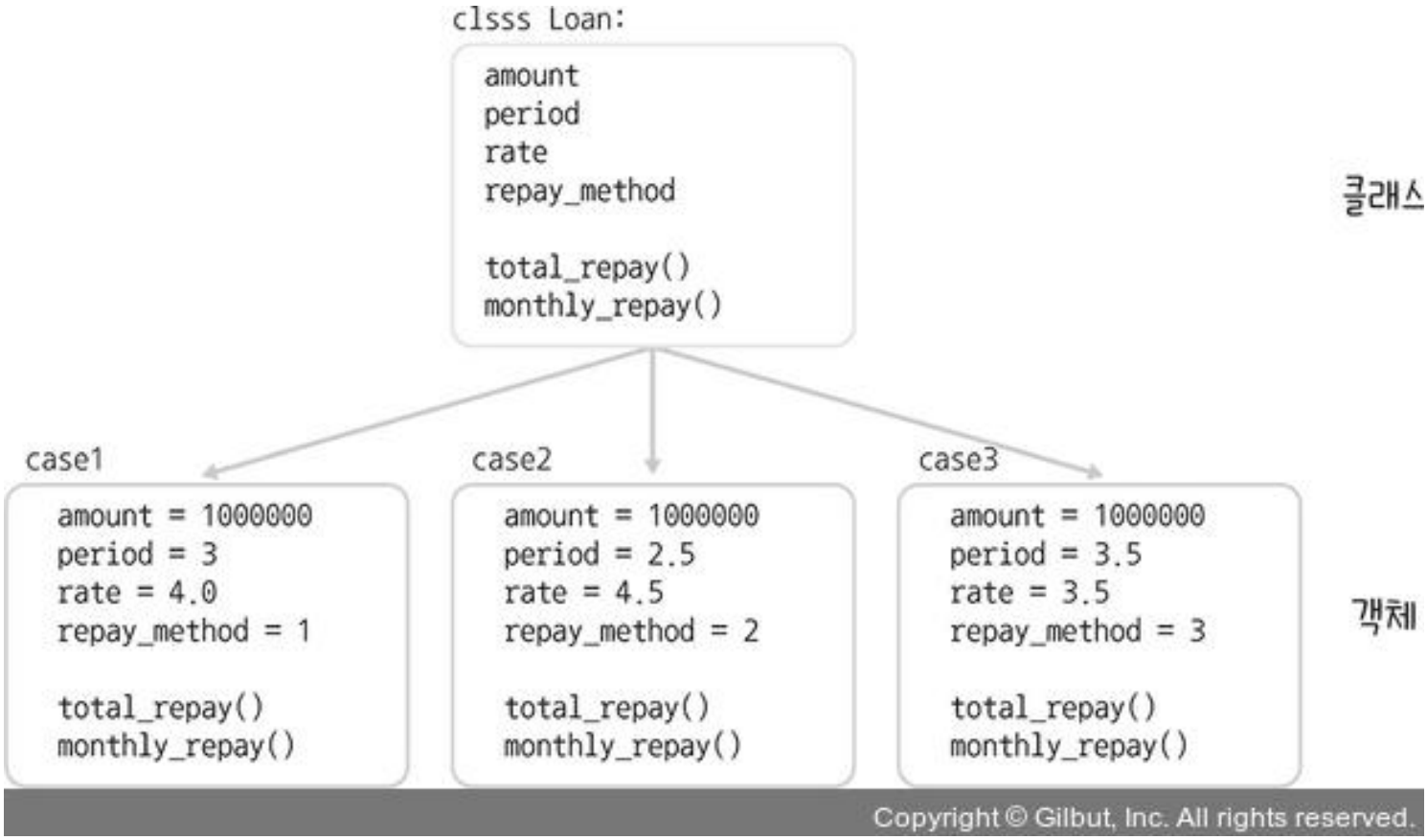
사용자 정의 함수를 사용해 조건별로 총 상환 금액과 월 상환 금액을 비교해 봅시다. 총 상환 금액을 계산하는 함수와 월 상환 금액을 계산하는 함수가 각각 하나씩 있어야죠. 각각 `total_repay`, `monthly_repay`로 정의한다면 대출 금액과 대출 금리, 상환 방식 등을 함수의 인자로 전달받아야 합니다. 이때 상환 방식은 원리금 균등 상환 = 1, 원금 균등 상환 = 2, 만기 일시 상환 = 3이라고 가정합니다. 조건별로 정리하면 다음과 같습니다.

▼ 표 14-2 사용자 정의 함수로 구현한 학자금 대출 조건

조건	사용자 정의 함수
조건 1	
	
조건 2	
	
조건 3	
	

각 조건을 비교하는 데 함수를 여섯 번 사용해야 합니다. 여기에 중도 상환 수수료 계산 등의 다른 기능을 추가하면 더 복잡해지겠죠.

반면에 클래스는 변수와 함수를 동시에 하나의 단위로 묶을 수 있습니다. 대출 금액, 대출 기간, 대출 금리, 상환 방식을 각각 저장하는 변수와 총 상환 금액을 계산하는 `total_repay` 함수, 월 상환 금액을 계산하는 `monthly_repay` 함수를 묶어 클래스로 만듭니다. 학자금 대출 조건을 클래스로 구성하면 다음과 같습니다. 클래스명은 `Loan`이라고 하겠습니다.



▲ 그림 14-9 클래스로 구현한 학자금 대출 조건

대출과 관련한 변수와 함수를 모두 `Loan` 클래스에 넣어 속성과 메서드로 만들었습니다. 이렇게 클래스를 정의하면 인스턴스 속성값을 바꿔 조건별 객체를 만들 수 있습니다. 여기서는 각 객체를 `case1`, `case2`, `case3`로 이름 붙였습니다. `case1`의 총 상환 금액을 알려면 `case1` 객체에서 `total_repay()` 메서드를 호출하고, `case3`의 월 상환 금액을 알려면 `case3` 객체에서 `monthly_repay()` 메서드를 호출하면 됩니다.

이처럼 클래스는 속성(변수)과 메서드(함수)를 하나로 묶어 다룬다는 점이 사용자 정의 함수와 다릅니다.

클래스와 객체는 이 책에서 배우는 내용 중 가장 복잡한 부분이라 이해하기 조금 어려울 수 있습니다. 하지만 한 번 이해하고 나면 높은 수준의 프로그래밍을 할 때 유용하니 시간이 조금 걸리더라도 꼭 자신의 것으로 만들어 보세요.

1분 퀴즈

1 다음 코드를 보고 틀린 설명을 고르세요.

```
class Student:
    def __init__(self, name, kor, eng, mat):
        self.name, self.kor, self.eng, self.mat = name, kor, eng, mat
    def score_avg(self):
        (가)
        print(self.name, '학생의 국영수 평균점수는', self.avg)

Jordy = Student((나))
```

① `Student` 클래스의 객체를 만들 때 `__init__()` 메서드도 동시에 실행된다.

② `score_avg()`가 세 과목의 평균 점수를 구하는 메서드라고 할 때, (가)에 들어갈 코드는 `self.avg = (self.kor + self.eng + self.mat)/3`이다.

③ `Jordy`가 `Student` 클래스의 객체일 때 `Jordy`의 국어 점수를 100으로 수정하려면 `Jordy.kor = 100`을 실행한다.

④ 이름이 'Jordy'고 국영수 점수가 각각 '56, 91, 39'인 객체를 만든다면 (나)에 들어갈 코드는 `self, 'Jordy', 56, 91, 39`다.

⑤ `Jordy` 객체의 평균 점수를 출력하려면 `Jordy.score_avg()`를 실행한다.

정답 및 해설

14.2

클래스와 객체로 프로그램 작성하기

클래스와 객체의 기본 구조와 사용법을 배웠습니다. 이번에는 클래스와 객체를 사용해 실생활 사례를 프로그램으로 작성해 보겠습니다.

온라인 쇼핑을 생각해 봅시다. 온라인 쇼핑몰에는 매우 다양한 상품이 등록되어 있습니다. 사람들은 온라인 쇼핑몰에 회원가입을 하고 물건을 구매합니다. 이를 클래스와 객체의 개념에 대입해 상품과 회원이라는 클래스를 만들어 보겠습니다. 그리고 새로운 상품을 등록할 때마다 상품 객체를 만들고, 새로운 회원이 가입할 때마다 회원 객체를 만듭니다.

14.2.1 상품 클래스 만들기

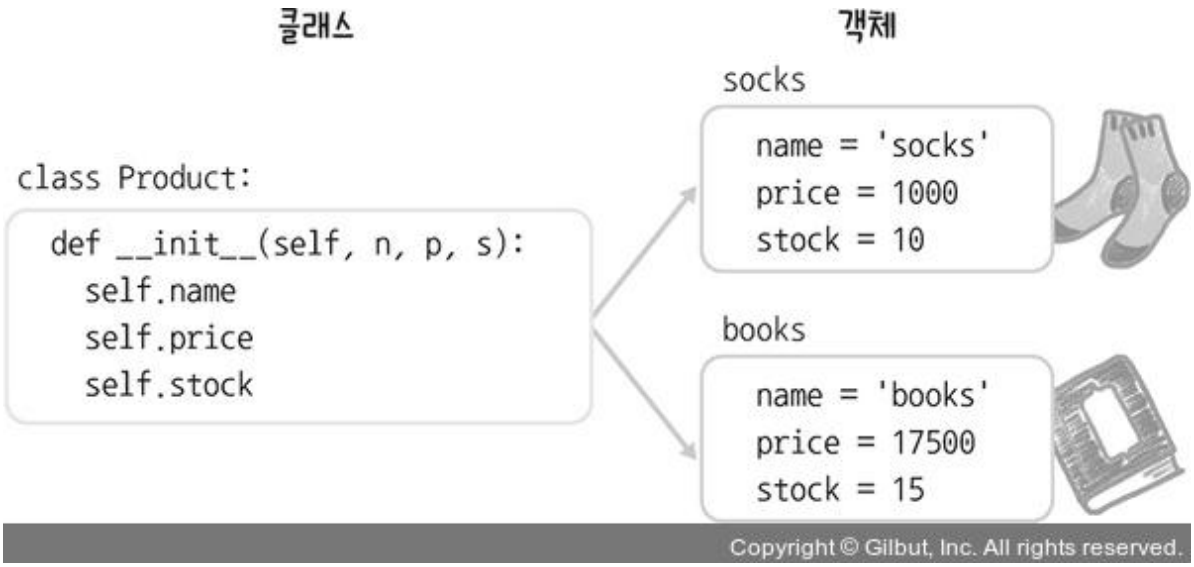
상품 클래스부터 구상해 봅시다. 온라인 쇼핑몰에 등록되는 상품들이 반드시 가져야 할 속성에는 무엇이 있을까요? 당연히 상품 이름과 가격이겠죠. 그리고 품절된 제품을 구매하지 않도록 재고수량도 보여 줍시다. 클래스명은 `Product`로 하겠습니다.

```
class Product:
  def __init__(self, n, p, s):
    self.name
    self.price
    self.stock
```

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 14-10 상품 클래스 구조

그림처럼 `Product` 클래스는 `name`(상품이름), `price`(상품가격), `stock`(재고수량)을 의미하는 인스턴스 속성이 있습니다. 그리고 새로운 상품을 등록할 때마다 `Product` 클래스를 바탕으로 하는 객체를 만들면 됩니다. 예를 들어, 양말을 쇼핑몰에 등록한다고 하면 `name`, `price`, `stock` 속성을 포함한 `socks` 객체를 생성합니다. 다른 상품도 마찬가지입니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 14-11 상품 클래스와 객체 구조

`Product` 클래스부터 정의해 봅시다.

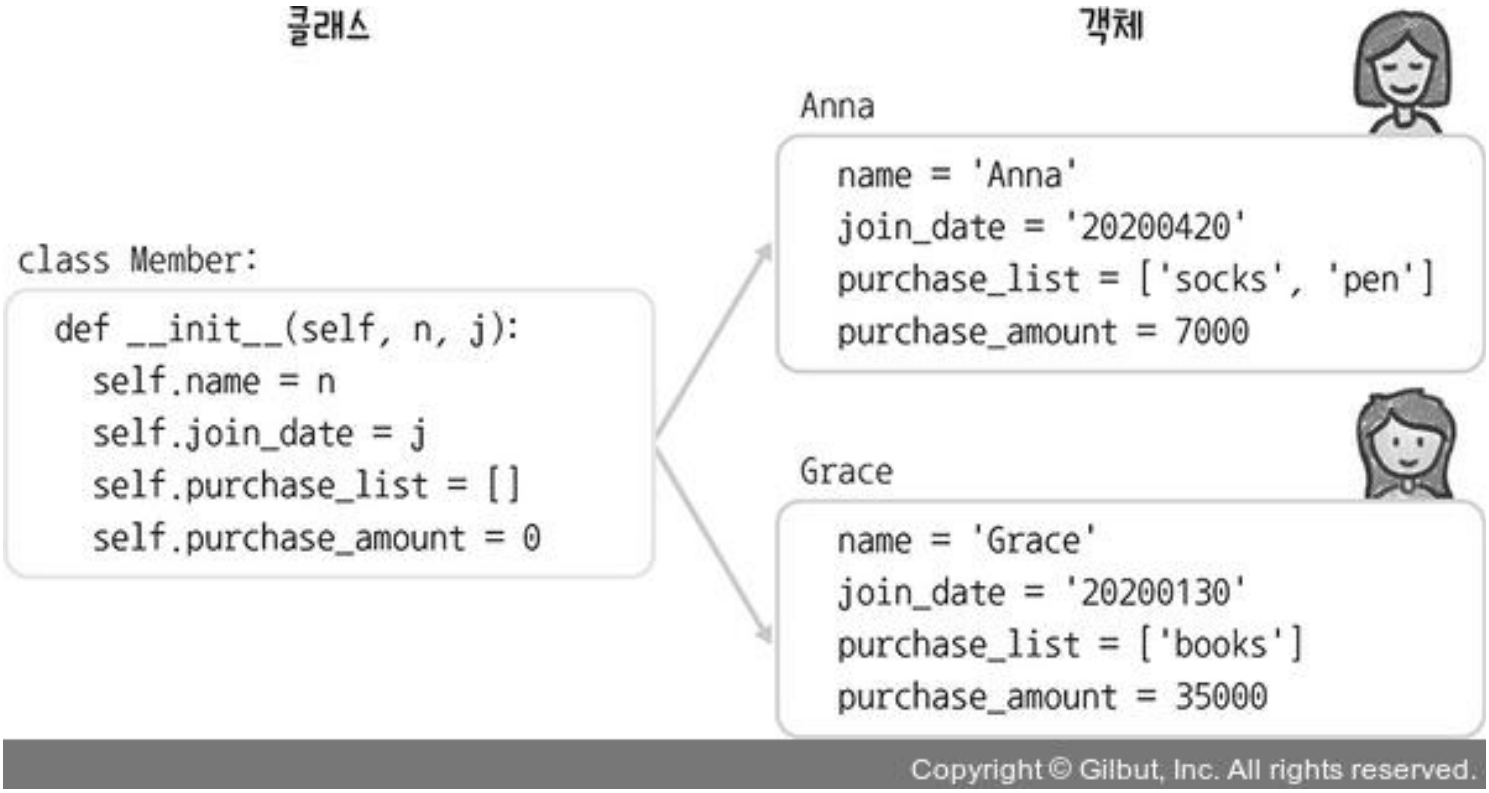
```
class Product:
    def __init__(self, n, p, s):
        self.name, self.price, self.stock = n, p, s

    def information(self):
        print('상품이름 :', self.name)
        print('상품가격 :', self.price)
        print('재고수량 :', self.stock)
```

`Product` 클래스에 생성자 `__init__()` 메서드를 작성했습니다. `__init__()` 메서드는 객체를 만들 때 상품이름, 상품가격, 재고수량을 전달받아 각 객체의 인스턴스 속성(`self.name`, `self.price`, `self.stock`)에 저장합니다. 상품의 정보를 알려주는 메서드 `information()`도 추가했습니다. 이렇게 상품 클래스가 완성됐습니다.

14.2.2 회원 클래스 만들기

회원 클래스도 구상해 봅시다. 회원 클래스에 있어야 할 기본 속성은 무엇일까요? 우선 회원이름이 있어야 합니다. 가입날짜도 있으면 좋으니 추가하죠. 그리고 구매내역과 누적 구매금액도 속성으로 넣어 봅시다. 고객이 직접 자신의 정보를 확인하는 경우도 있고, 쇼핑몰 관리자가 고객 소비 패턴을 분석할 때 유용하게 쓰일 수 있습니다.



▲ 그림 14-12 회원 클래스와 객체 구조

회원 클래스인 `Member`를 작성해 봅시다. `Member` 클래스는 회원이름과 가입날짜만 전달받아 각 인스턴스 속성에 저장합니다. 구매내역은 여러 개의 상품이 추가될 수 있으므로 리스트로 만듭니다. 구매내역과 누적 구매금액은 회원가입 시점(객체 생성 시)에는 아무것도 없는 상태이므로 각각 초깃값으로 빈 리스트와 0을 넣어 줍니다. 그리고 회원의 기본정보를 출력하는 `information()` 메서드도 추가로 작성합니다.

```
class Member:
    def __init__(self, n, d):
        self.name, self.join_date = n, d
        self.purchase_list = []
        self.purchase_amount = 0

    def information(self):
        print('회원이름 :', self.name)
        print('가입날짜 :', self.join_date)
        print('구매내역 :', self.purchase_list)
        print('누적 구매금액 :', self.purchase_amount)
```

속성에 값을 저장하고, 이를 출력하는 것이 전부이기 때문에 어려운 부분은 없습니다.

다음으로 `Member` 클래스에 있어야 하는 기능을 생각해 봅시다. 쇼핑몰에서 회원이 가장 많이 하는 일은 구매죠. 따라서 상품을 구매하는 메서드가 있어야 합니다. 메서드명은 `buy`로 하겠습니다. 상품 구매 시 일어나는 일을 생각해 봅시다. 회원이 상품을 결제하면 회원의 구매내역(`purchase_list`)에 상품이름이 추가되고, 누적 구매금액(`purchase_amount`)에 결제금액만큼 더해져야 합니다. 회원이 가진 속성을 제대로 변경했다면 팔린 상품의 재고수량을 팔린 개수만큼 차감해야 합니다.

구매내역에 구매한 상품을 추가하려면 상품이름을 알아야죠. 누적 구매금액을 변경하려면 상품가격과 구매수량을 알아야 하고요. 상품이름과 가격은 `Member` 클래스에서는 알 수가 없고, 어떤 상품을 사느냐에 따라 달라집니다. 따라서 상품이름과 가격은 인자로 전달받아야 합니다. `Product` 객체를 인자로 전달받으면 그 객체의 속성(상품이름, 가격)에 접근할 수 있겠죠. 구매수량도 그때그때 다르므로 클래스 외부에서 전달받도록 매개변수 `count`를 사용하겠습니다.

```
class Member:
    def __init__(self, n, d):
        self.name, self.join_date = n, d
        self.purchase_list = []
        self.purchase_amount = 0

    def information(self):
        print('회원이름 :', self.name)
        print('가입날짜 :', self.join_date)
        print('구매내역 :', self.purchase_list)
        print('누적 구매금액 :', self.purchase_amount)

    # Product 객체가 인자로 전달됨
    def buy(self, product, count):
        print(self.name, '고객님이', product.name, count, '개 구매!')
        product.stock -= count # 구매수량만큼 재고 감소
        self.purchase_list.append(product.name)         # 구매내역에 상품 추가
        self.purchase_amount += (product.price * count) # 누적 구매금액 증가
```

`Product` 클래스의 객체가 매개변수 `product`로 전달되면 상품이름과 상품가격은 `product.name`과 `product.price`로 접근할 수 있습니다. 따라서 구매내역에 상품이름을 추가할 수 있습니다. 상품 구매수량은 `count`로 전달되므로 `Product` 객체의 속성인 재고(`product.stock`)도 구매수량만큼 감소시킬 수 있습니다. 총 구매금액은 상품가격(`product.price`) × 구매수량(`count`)이 되고, 이를 이용해 누적 구매금액도 수정합니다. `Member` 클래스도 완성했습니다.

14.2.3 상품등록, 회원가입, 물품구매 코드 작성하기

실제 쇼핑몰처럼 상품을 등록하고, 회원가입도 해 보겠습니다. 상품은 양말(1,000원, 10개)과 책(17,500원, 15개)을, 회원은 Anna(20200420 가입)와 Grace(20200130 가입)를 등록합니다.

```
# 물건 등록
socks = Product('socks', 1000, 10)
books = Product('books', 17500, 15)

# 회원가입
Anna = Member('Anna', '20200420')
Grace = Member('Grace', '20200130')
```

물품 구매 행위를 구현해 봅시다. Anna가 양말 1개를 구매한다면 다음 코드처럼 작성할 수 있습니다.

```
Anna.buy(socks, 1)
```

```
실행결과
Anna 고객님이 socks 1 개 구매!
```

`Anna.buy(socks, 1)`로 Anna가 양말을 구매한 행위를 표현합니다.

현재 `Anna` 객체의 정보를 출력해 봅시다.

```
Anna.information()
```

```
실행결과
고객이름 : Anna
가입날짜 : 20200420
구매내역 : ['socks']
누적 구매금액 : 1000
```

구매내역과 누적 구매금액에 새로운 값이 저장됐습니다. 구매한 양말은 구매내역(`Anna.purchase_list`)에, 결제금액 1,000원(1,000원 × 1개)은 누적 구매금액(`Anna.purchase_amount`)에 저장됐습니다. `socks` 객체의 정보도 출력해 봅시다.

```
socks.information()
```

```
실행결과
상품이름 : socks
상품가격 : 1000
재고수량 : 9
```

이번에는 Grace가 책을 2권 샀다고 가정하고 코드를 작성해 봅시다.

```
Grace.buy(books, 2)
books.information()
```

```
실행결과
Grace 고객님이 books 2 개 구매!
상품이름 : books
상품가격 : 17500
재고수량 : 13
```

상품을 구매하면 `Grace` 객체가 `buy()` 메서드를 실행하고 다음으로 `books` 객체가 `information()` 메서드를 실행합니다. 책 2권을 구매했으므로 재고수량은 13권이 됩니다.

그럼 `Grace` 객체의 정보를 조회해 봅시다.

```
Grace.information()
```

```
실행결과
고객이름 : Grace
가입날짜 : 20200130
구매내역 : ['books']
누적 구매금액 : 35000
```

구매내역과 누적 구매금액에 값이 저장됐습니다. 책 한 권이 17,500원이므로 누적 구매금액(`product.price * count`)은 35,000원이 됩니다.

이번에는 새로운 상품인 펜(1,200원, 25개)을 등록하고, Anna가 펜을 5개 샀다고 가정해 봅시다. 이제 클래스와 객체에 익숙해졌을테니 책을 확인하기 전에 먼저 코드를 직접 작성해 보세요.

```
# 새로운 물건 입고
pen = Product('pen', 1200, 25)

Anna.buy(pen, 5)
```

```
실행결과
Anna 고객님이 pen 5 개 구매!
```

새로운 상품이 등장했으니 `Product` 클래스의 새로운 객체를 만듭니다. 객체명은 `pen`으로 하고, 상품이름, 상품가격, 재고수량을 차례대로 인자로 넣어 객체를 만듭니다. 그리고 Anna가 펜 5개를 구매하면 남은 펜의 개수를 5개 줄입니다.

Anna의 정보를 확인해 봅시다.

```
Anna.information()
```

```
실행결과
고객이름 : Anna
가입날짜 : 20200420
구매내역 : ['socks', 'pen']
누적 구매금액 : 7000
```

구매내역 리스트의 요소가 2개가 됐습니다. 앞서 구매한 양말 뒤에 펜이 추가됐죠. `Anna` 객체의 `purchase_list` 속성이 사라지지 않고 계속 유지되는 것을 알 수 있습니다. 누적 구매금액 역시 1,000원에서 7,000원으로 증가했습니다. 펜 구매금액(1,200원 × 5개)이 추가된 결과입니다.

이처럼 클래스와 객체는 자주 반복되는 단위를 표현하고 재사용하기에 좋습니다. 또한, 클래스와 객체를 잘 활용하면 프로그래밍의 직관성과 효율성을 동시에 높일 수 있습니다.

14.3

클래스와 객체로 핫플레이스 프로젝트 재구성하기

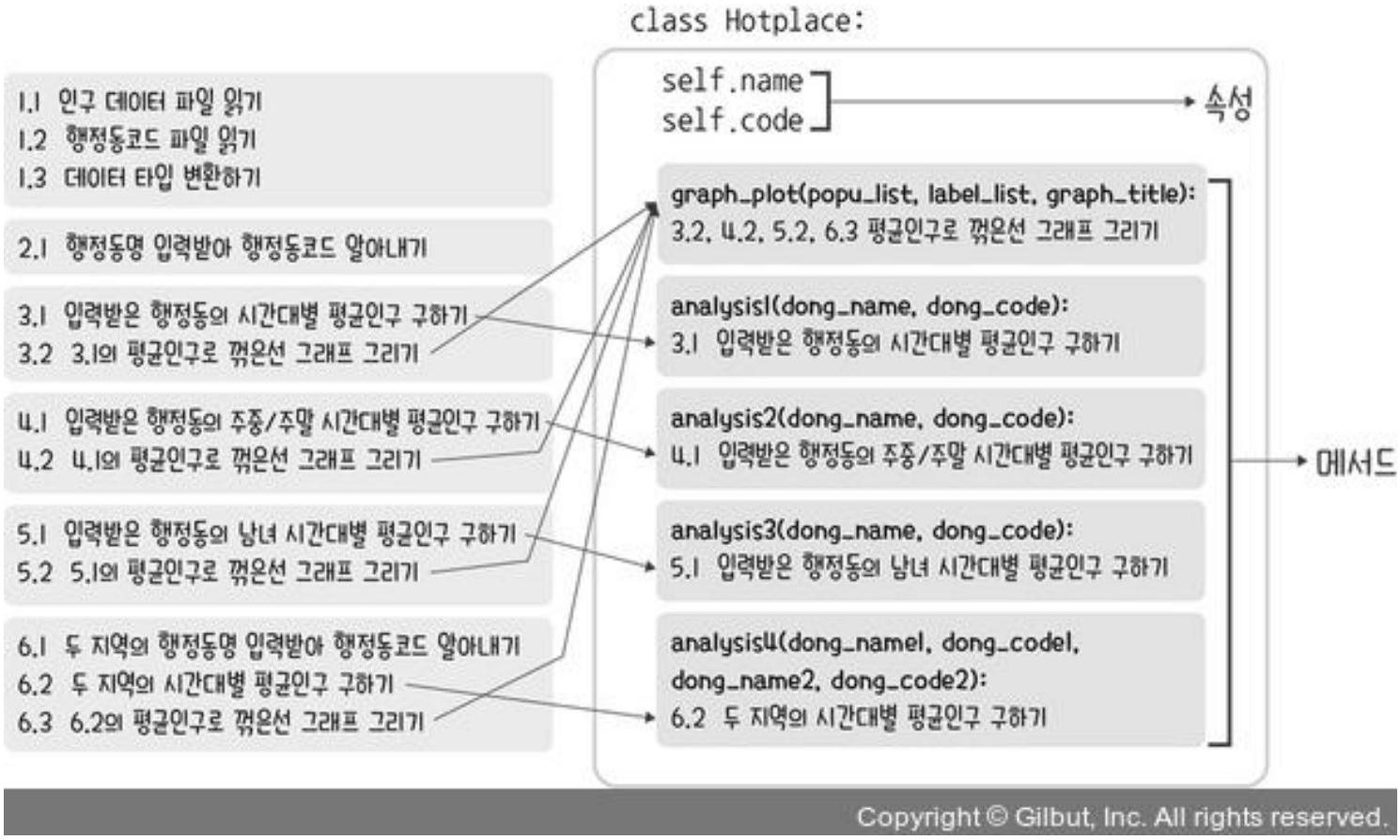
13장에서 사용자 정의 함수로 핫플레이스 프로젝트를 재구성했습니다. 그 결과로 중복 코드가 제거되고 전체 코드도 직관적으로 변했습니다. 이번에는 클래스와 객체로 핫플레이스 프로젝트를 재구성해 봅시다. 앞 장에서 완성한 코드를 바탕으로 더욱 발전시켜 보겠습니다.

14.3.1 클래스 구조 만들기

핫플레이스 프로젝트를 어떻게 클래스와 객체로 재구성할 수 있을까요? 핫플레이스 프로젝트는 행정동명과 행정동코드로 해당 행정동의 인구를 분석합니다. 따라서 행정동명과 행정동코드를 속성으로, 인구 분석 기능을 메서드로 가진 핫플레이스 클래스를 만들면 됩니다.

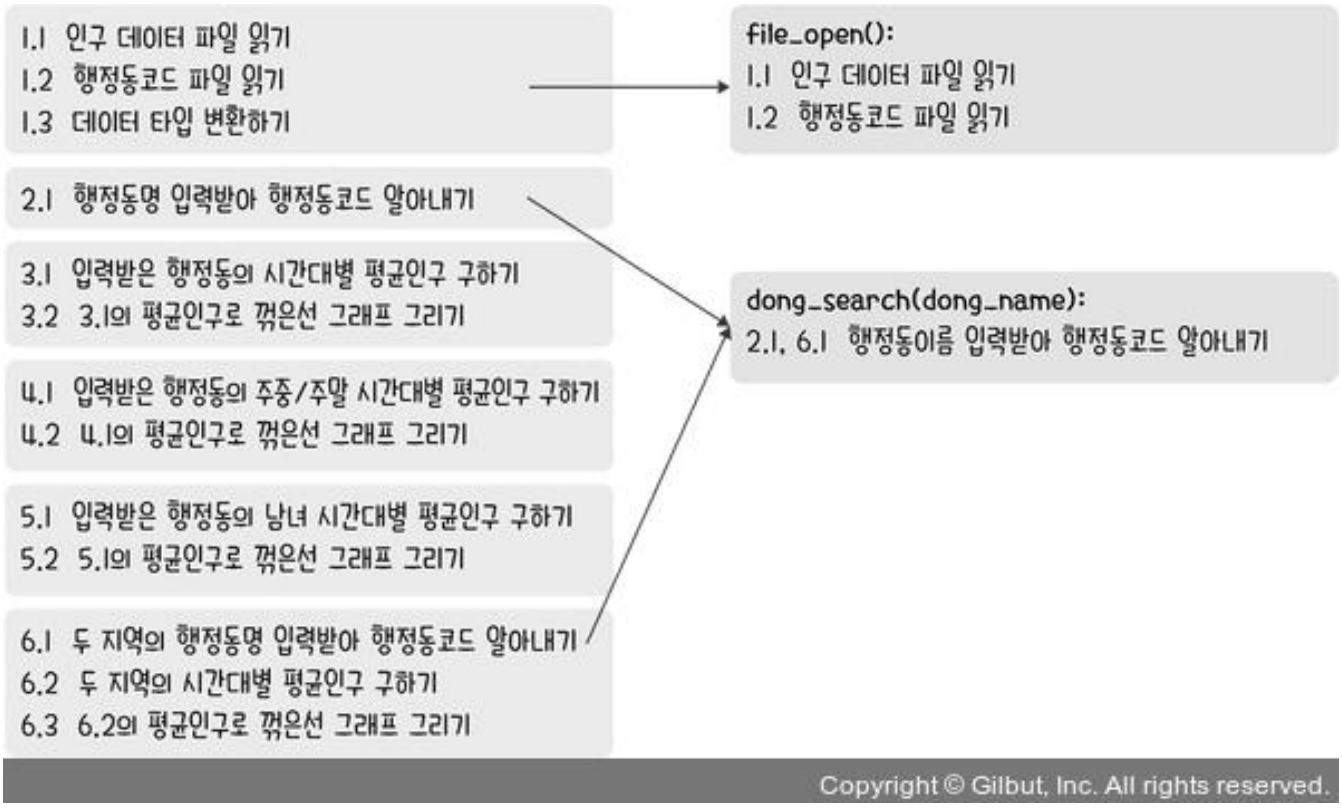
클래스명은 `Hotplace`로 정합니다. 클래스의 속성은 인구 분석을 진행할 행정동명과 행정동코드이므로 행정동명은 `name`, 행정동코드는 `code`로 합니다. 클래스의 메서드는 13장에서 인구를 분석한 사용자 정의 함수 `analysis1()`, `analysis2()`, `analysis3()`, `analysis4()`를 그대로 사용하면 되고요. 클래스의 메서드에는 매개변수로 `self`만 사용합니다. 인자로 전달받는 각 객체에 이미 `self.name`과 `self.code`가 있으니까요.

`Hotplace` 클래스의 구조를 정리하면 다음과 같습니다.



▲ 그림 14-13 Hotplace 클래스의 구조

인구 분석 함수는 모두 그래프를 그립니다. 그래서 모든 인구 분석 함수는 마지막에 그래프를 그리는 함수인 `graph_plot()`을 호출합니다. 그리고 `file_open()`과 `dong_search()` 함수도 있었죠. 객체를 생성하기 전에 미리 `file_open()` 함수로 필요한 데이터 파일을 읽고, `dong_search()` 함수로 행정동명과 행정동코드를 알아내 `Hotplace` 클래스의 객체를 생성할 때 인자로 전달해야 합니다. 따라서 `Hotplace` 클래스에 포함하지 않고 클래스 외부에 둡니다.



▲ 그림 14-14 Hotplace 클래스 외부 함수

핫플레이스 프로젝트를 클래스로 재구성하면 이와 같습니다. 그럼 코드를 직접 수정해 봅시다.

14.3.2 코드 수정하기

`Hotplace` 클래스에는 행정동명과 행정동코드를 저장하는 속성이 필요합니다. 따라서 객체를 생성하면 자동으로 전달받은 행정동명과 행정동코드를 인스턴스 속성에 저장하도록 생성자 `__init__()` 메서드에 추가합니다. 생성자에서는 행정동명과 행정동코드를 매개변수 `dong_name`과 `dong_code`로 받아 인스턴스 속성 `self.name`과 `self.code`에 저장합니다.

`Hotplace` 클래스에는 6개의 메서드가 있습니다. 생성자 `__init__()`과 `graph_plot()` 그리고 `analysis1()`~`analysis4()`입니다. 모든 메서드에 첫 번째 매개변수로 `self`를 추가하고, 각 메서드 내부에서 사용하는 `dong_name`과 `dong_code`는 인스턴스 속성인 `self.name`, `self.code`로 변경합니다.

`graph_plot()`과 `analysis2()`, `analysis3()`부터 `Hotplace` 클래스의 메서드로 바꿔 봅시다.

```
import matplotlib.pyplot as plt
import datetime

class Hotplace:
    def __init__(self, dong_name, dong_code):
        self.name = dong_name
        self.code = dong_code

    # 그래프 그리기 메서드
    def graph_plot(self, popu_list, label_list, graph_title):
        plt.rc('font', family='Malgun Gothic')
        plt.title(graph_title)
        for i in range(len(popu_list)):
            plt.plot(range(24), popu_list[i], label=label_list[i])
        plt.legend()
        plt.xlabel('시간대')
        plt.ylabel('평균인구수')
        plt.xticks(range(24), range(24))
        plt.show()

    # 하위 목표 1 부분

    # 하위 목표 2
    def analysis2(self):
        weekday = [0 for i in range(24)]
        weekend = [0 for i in range(24)]
        for row in data:
            if row[2] == self.code:
                time, p = row[1], row[3]
                year, mon, day = int(row[0][:4]), int(row[0][4:6]), int(row[0][6:])
                num = datetime.date(year, mon, day).weekday()
                if num < 5:
                    weekday[time] += p
                else:
                    weekend[time] += p
        weekday_cnt, weekend_cnt = 0, 0
        for i in range(1, 32):
            if datetime.date(2019, 12, i).weekday() < 5:
                weekday_cnt += 1
            else:
```

```
                weekend_cnt += 1
        weekday = [w/weekday_cnt for w in weekday]
        weekend = [w/weekend_cnt for w in weekend]
        data_set = [weekday, weekend]
        labels = ['주중', '주말']
        title = self.name + ' 주중/주말 시간대별 평균인구'
        self.graph_plot(popu_list = data_set, label_list = labels, graph_title = title)

    # 하위 목표 3
    def analysis3(self):
        male = [0 for i in range(24)]
        female = [0 for i in range(24)]
        for row in data:
            if row[2] == self.code:
                time = row[1]
                male[time] += sum(row[4:18])
                female[time] += sum(row[18:32])
        male = [m/31 for m in male]
        female = [f/31 for f in female]
        data_set = [male, female]
        labels = ['남성', '여성']
        title = self.name + ' 남녀 시간대별 평균인구'
        self.graph_plot(popu_list = data_set, label_list = labels, graph_title = title)

# 하위 목표 4 부분
```

코드가 길지만, 앞에서 여러 번 다뤘으므로 익숙할 겁니다. 바뀐 부분만 따로 표시했습니다. 생성자 `__init__()` 부분을 추가하고 속성과 메서드를 사용할 때 앞에 `self`를 붙였습니다.

나머지 `analysis1()`과 `analysis4()` 메서드를 봅시다. `analysis1()` 메서드만 보면 `analysis1(self)`로 변경하면 됩니다. 그런데 `analysis4()` 메서드에는 두 지역을 비교하기 위해 `analysis1()` 메서드를 호출하는 부분이 있죠. `analysis1(dong_name, dong_code)`에서 매개변수 `dong_name`, `dong_code`를 삭제하고 `analysis1(self)`로 작성하면 핫플레이스가 있는 지역의 평균인구는 `self.name`, `self.code`로 구할 수 있습니다. 하지만 비교하려는 지역의 평균인구는 구할 수 없습니다. 매개변수가 `self` 하나여서 비교하려는 지역의 행정동명과 코드를 전달받을 수 없기 때문이죠. 따라서 행정동명과 행정동코드를 전달받는 매개변수를 `analysis1()`에 그대로 두어야 합니다. 물론 가장 앞에 객체를 전달받는 `self`를 추가하고요.

`analysis1()` 메서드를 작성해 봅시다(하위 목표 1 부분에 추가하면 됩니다).

```
# 하위 목표 1
# analysis4()에서 다른 지역을 넣어 호출하는 부분이 있어 매개변수를 그대로 두어야 함
def analysis1(self, dong_name, dong_code, graph_yn = 'y'):
    population = [0 for i in range(24)]
    for row in data:
        if row[2] == dong_code:
            time, p = row[1], row[3]
            population[time] += p
    population = [p/31 for p in population]
    if graph_yn == 'n':
        return population
    population = [population]
    labels = ['평균인구']
    title = dong_name + ' 시간대별 평균인구'
    self.graph_plot(popu_list=population, label_list=labels, graph_title=title)
```

`analysis1()` 메서드에서는 매개변수 `self`를 추가하고, `graph_plot()` 메서드를 호출할 때 앞에 `self`를 추가하는 부분만 수정하면 됩니다.

마지막으로 `analysis4()` 메서드를 작성해 봅시다(하위 목표 4 부분에 추가하면 됩니다). `analysis4(dong_name1, dong_code1)` 함수에서 매개변수 `dong_name1`, `dong_code1`은 핫플레이스의 행정동명과 행정동코드죠. 인스턴스 속성 `self.name`, `self.code`에 저장된 상태이므로 두 속성으로 대체하면 됩니다. 그리고 11장에서 비교할 지역의 행정동명과 코드는 `analysis4()` 함수 내부에서 `dong_search()`를 활용해 얻었습니다.

그런데 앞에서 클래스로 재구성할 때 `dong_search()` 함수를 클래스 외부에 정의하기로 했습니다. 따라서 비교할 행정동명과 코드는 클래스 외부에서 전달받아야 합니다. 따라서 `Hotplace` 클래스 내부에 정의하는 `analysis4()` 메서드는 비교할 행정동명과 코드를 전달받을 매개변수가 필요합니다. 이 매개변수를 각각 `dong_name2`, `dong_code2`라고 하겠습니다. `dong_name2`, `dong_code2`를 전달받으면 해당 지역의 평균인구를 구하는 부분부터는 동일합니다.

```
# 하위 목표 4
def analysis4(self, dong_name2, dong_code2):
    population1 = self.analysis1(dong_name = self.name, dong_code = self.code, graph_yn =
'n')
    population2 = self.analysis1(dong_name = dong_name2, dong_code = dong_code2, graph_yn =
'n')
    data_set = [population1, population2]
    labels = [self.name, dong_name2]
    title = self.name + '과 ' + dong_name2 + ' 시간대별 평균인구'
    self.graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

이렇게 기존 핫플레이스 프로젝트를 `Hotplace` 클래스로 변경했습니다. 클래스에 포함되지 않는 나머지 함수 부분은 그대로 유지하므로 기존 코드를 그대로 복사해서 붙여넣기합니다. 파이썬 파일이 있는 폴더에 인구 데이터 파일(LOCAL_PEOPLE_DONG_201912.csv)과 행정동코드 파일(dong_code.csv)을 넣는 것도 잊지 마세요!

```
import csv

def file_open():
    f = open('LOCAL_PEOPLE_DONG_201912.csv', encoding = 'utf8') # 또는 'eur-kr', 'cp949'
    data = csv.reader(f)
    next(data)
    data = list(data)

    f2 = open('dong_code.csv', encoding = 'cp949') # 또는 'eur-kr', 'utf8'
    code_data = csv.reader(f2)
    next(code_data)
    next(code_data)
    code_data = list(code_data)

    for row in data:
        for i in range(1, 32):
            if i <= 2:
                row[i] = int(row[i])
            else:
                row[i] = float(row[i])
    for row in code_data:
        row[1] = int(row[1])
    return data, code_data

def dong_search(dong_name):
    for row in code_data:
        if row[-1] == dong_name:
            code = row[1]
    print(dong_name, '-', code, '를 분석합니다!')
    return code
```

클래스와 외부 함수까지 준비됐습니다. 그럼 수정한 프로그램으로 인구 분석을 진행해 봅시다. 인구 분석에 필요한 파일을 읽고, 분석할 행정동을 입력받는 부분이 가장 먼저 실행돼야 하죠. 행정동을 입력받아 `input_name1`에 저장하고, 이를 행정동코드로 변환해 `input_code1`에 저장하겠습니다. 이 장에서는 성동구의 행당1동과 행당2동을 입력받아 인구 데이터를 비교합니다.

```
data, code_data = file_open()
input_name1 = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code1 = dong_search(input_name1)
```

```
실행결과
핫플레이스가 위치한 행정동을 입력하세요. --> 행당1동
행당1동 - 11200560 을(를) 분석합니다!
```

행당1동을 입력하니 `input_name1`, `input_code1`에 행정동명과 행정동코드가 각각 저장됐습니다. 행당1동의 행정동코드를 알았으니 `Hotplace` 클래스의 객체를 만들어 봅시다. 객체명은 `hangdang1`로 하겠습니다.

```
hangdang1 = Hotplace(input_name1, input_code1)
```

객체를 생성할 때 행당1동의 이름과 행정동코드가 저장된 변수를 인자로 전달하면 `__init__()` 메서드에서 `self.name`과 `self.code`에 자동으로 저장됩니다. 이렇게 `Hotplace`의 첫 번째 객체 `hangdang1`이 만들어집니다.

행당2동의 객체도 만들어 봅시다.

```
input_name2 = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code2 = dong_search(input_name2)
hangdang2 = Hotplace(input_name2, input_code2)
```

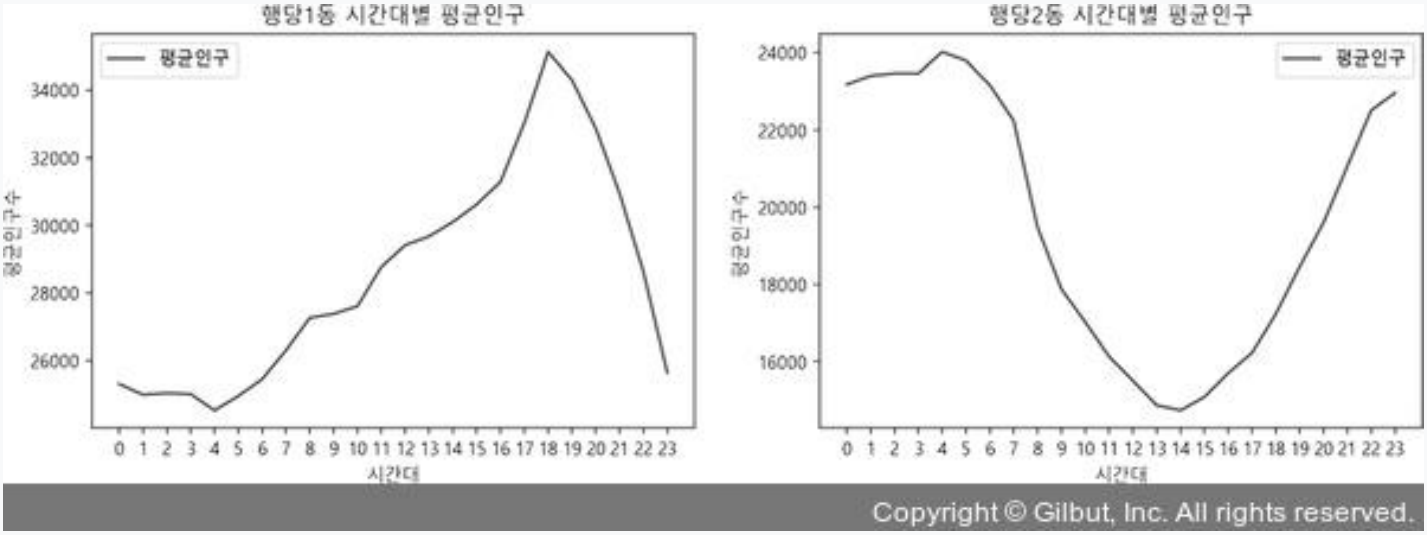
```
실행결과
핫플레이스가 위치한 행정동을 입력하세요. --> 행당2동
행당2동 - 11200570 을(를) 분석합니다!
```

행당2동의 행정동명과 행정동코드를 `input_name2`, `input_code2`에 저장해 `Hotplace` 클래스의 생성자에 전달했습니다. 이제 행당1동을 분석하려면 `hangdang1` 객체, 행당2동을 분석하려면 `hangdang2` 객체에서 필요한 메서드를 실행하면 됩니다.

행당1동과 행당2동의 평균인구 그래프를 그려 보겠습니다.

```
hangdang1.analysis1(input_name1, input_code1)
hangdang2.analysis1(input_name2, input_code2)
```

```
실행결과
```



Copyright © Gilbut, Inc. All rights reserved.

`hangdang1`과 `hangdang2` 객체에서 각각 `analysis1()` 메서드를 실행합니다. `analysis1()` 메서드는 행정동명과 행정동코드가 필요하므로 `input_name1`, `input_code1`과 `input_name2`, `input_code2`를 각각 인자로 전달합니다.

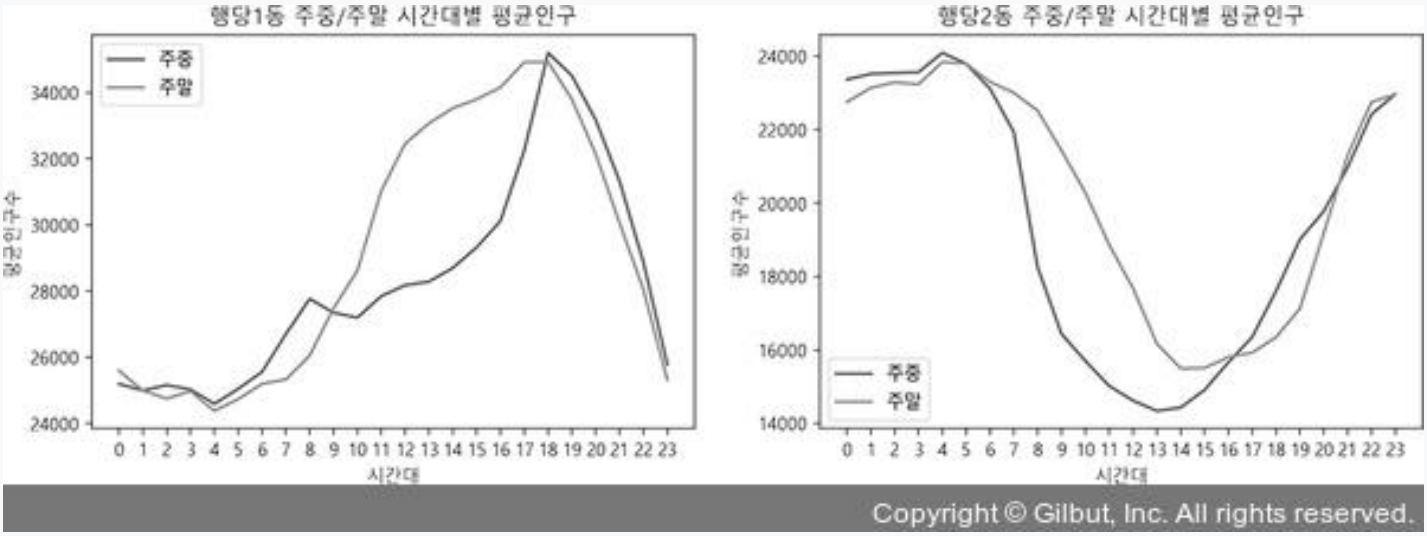
실행결과로 그래프 2개가 그려집니다. 행당1동과 행당2동은 인접한 동네지만, 인구 패턴이 완전히 다릅니다. 행당1동은 출근시간 즈음부터 인구가 증가하다가 퇴근시간 무렵에 인구가 가장 많습니다. 반면에 행당2동은 전형적인 주거지구의 인구 패턴을 보입니다. 아침과 저녁에 인구가 가장 많고 일과 시간에 인구가 적습니다.

행당1동을 지도에서 검색해 보니 왕십리역이 있습니다. 왕십리역은 2호선, 5호선, 분당선, 경의중앙선 총 4개의 도시철도가 지나가는 교통의 요지죠. 교통이 편리해서 약속을 잡기에 좋은 장소가 아닐까요? 그리고 교통이 발달하면 상권도 자연스럽게 형성되므로 놀거리와 먹거리가 많습니다. 행당2동을 검색하면 거의 대부분의 지역에 아파트가 분포합니다. 평균인구 그래프가 실행결과처럼 나온 이유가 이해됩니다.

이번에는 행당1동과 행당2동의 주중/주말 인구를 비교해 봅시다.

```
hangdang1.analysis2()
hangdang2.analysis2()
```

실 행 결 과



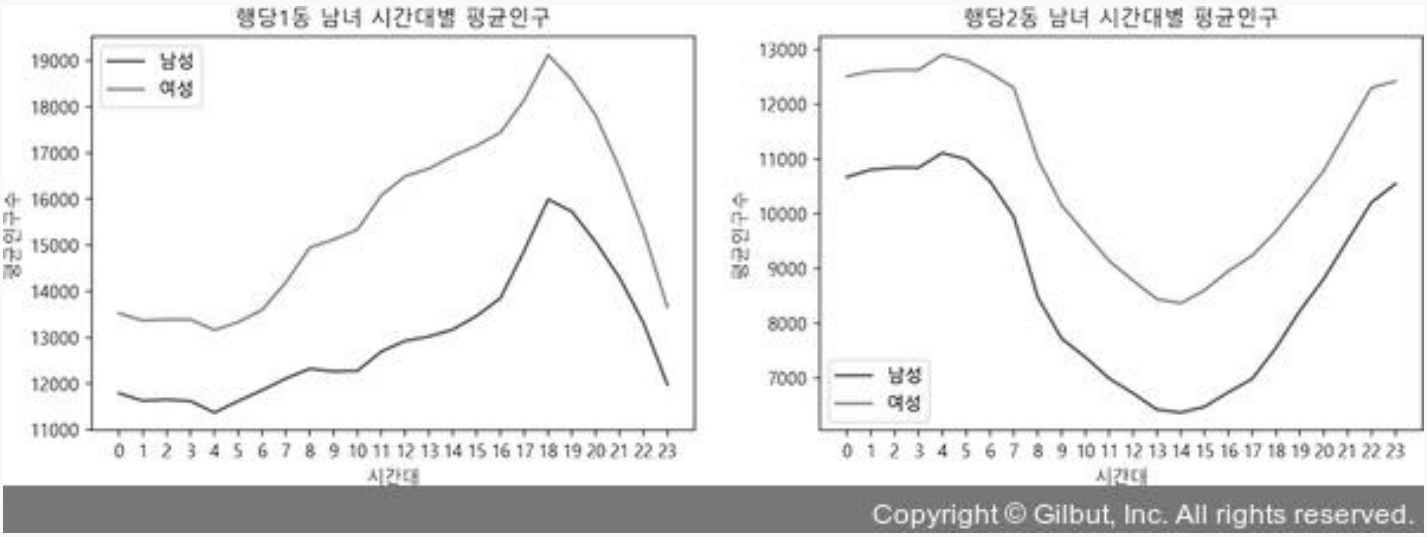
Copyright © Gilbut, Inc. All rights reserved.

`analysis2()` 메서드는 인자를 전달하지 않아도 되므로 메서드명만으로 호출하면 됩니다. 출력된 그래프를 보니 행당1동과 행당2동은 주중, 주말 인구에서도 차이가 있습니다. 행당1동은 오전 9시부터 오후 5시 사이에 주말 인구가 더 많습니다. 왕십리역에서 환승하거나 약속을 잡는 인구가 주말 낮에 많다고 분석할 수 있습니다. 행당2동은 오전 6시부터 오후 4시까지 주말인구가 더 많은 것으로 나옵니다. 주거지구이므로 아무래도 주말에는 느긋하게 집에서 쉬다가 오후에 외출하는 것이 아닐까요?

성별 인구 그래프도 비교해 봅시다. `analysis3()` 역시 인자를 넣어 주지 않아도 됩니다.

```
hangdang1.analysis3()
hangdang2.analysis3()
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

행당1동과 행당2동 모두 남녀 시간대별 인구 변화는 유사합니다. 그런데 항상 여성 생활인구가 더 많습니다. 행당동은 여성 생활인구가 더 많은 지역이군요.

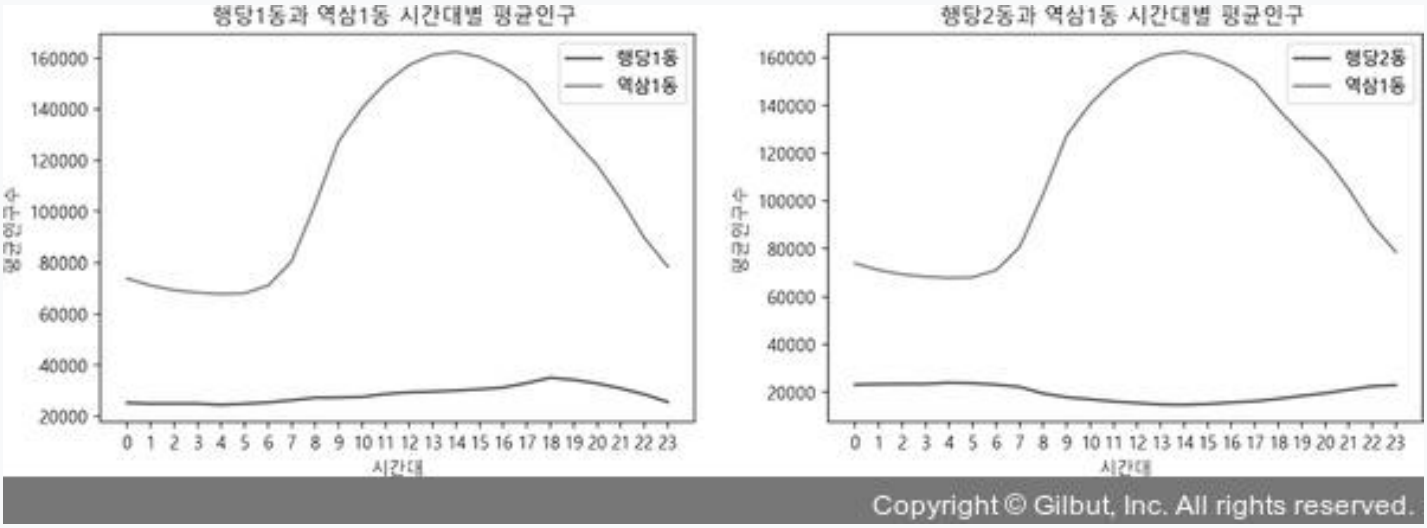
이번에는 행당1동, 행당2동을 다른 지역과 비교해 보겠습니다. 서울에서 인구가 많은 지역 중 하나인 역삼1동과 비교해 봅시다. 먼저 `dong_search()` 함수로 역삼1동의 행정동코드를 조회해야죠. 그러고 나서 역삼1동의 이름과 행정동코드를 `input_name3`, `input_code3`에 저장합니다. 행당1동과 역삼1동의 비교하려면 `hangdang1` 객체에서 `analysis4()` 메서드를 실행합니다. 행당2동과 역삼1동의 비교하려면 `hangdang2` 객체에서 `analysis4()` 메서드를 실행합니다.

```
input_name3 = input('비교할 행정동을 입력하세요. --> ')
input_code3 = dong_search(input_name3)

hangdang1.analysis4(input_name3, input_code3)
hangdang2.analysis4(input_name3, input_code3)
```

실행결과

```
비교할 행정동을 입력하세요. --> 역삼1동
역삼1동 - 11680640 을(를) 분석합니다!
```



결과를 보면 역삼1동의 인구가 행당1동과 행당2동보다 압도적으로 많네요. 역삼1동은 인구 변화도 커서 행당동의 인구 변화가 미미하게 느껴질 정도입니다.

핫플레이스 프로젝트를 클래스와 객체로 재구성하니 동시에 다양한 지역을 비교할 수 있게 됐습니다. 이 장 초반에 여러 대출 조건을 객체로 만들어 비교하는 예시로 클래스와 객체의 이점을 설명했는데, 핫플레이스 프로젝트에도 동일하게 적용할 수 있습니다. 지역별로 각각 객체를 만들 수 있으니 다양한 지역을 동시에 비교할 수 있습니다. 다른 궁금한 지역이 있다면 객체를 생성해 분석해 보세요!

14

마무리

1 클래스 정의하기

형식

```
class 클래스명:
    속성1                # 클래스 속성
    속성2                # 클래스 속성
    def __init__(self): # 생성자: 객체 생성과 동시에 실행되는 메서드
        self.속성1       # 인스턴스 속성
        self.속성2       # 인스턴스 속성
    def 메서드(self):
```

2 객체 생성하기

형식

```
객체명 = 클래스명()
```

3 객체의 속성 사용하기

형식

```
객체명.속성
```

4 객체에서 메서드 호출하기

클래스에 정의된 메서드는 `객체명.메서드()`로 호출한다. 호출 시 첫 번째 인자로 호출한 객체를 넘긴다. 따라서 클래스에서 메서드를 정의할 때는 첫 번째 매개변수로 `self`를 항상 넣어야 한다.

형식

```
객체명.메서드()
```

Self Check

고객계좌 클래스와 은행 클래스를 정의해 금융 거래 프로그램을 작성하려고 합니다. 다음은 각 클래스가 가져야 하는 조건과 완성된 클래스의 일부입니다. 내용을 보고 다음 물음에 답하세요.

|조건|

- 고객계좌 클래스(`Customer_account`)는 고객이름과 잔고를 속성으로 가지며, 메서드는 없습니다.
- 은행 클래스(`Bank`)는 3개의 메서드가 있습니다. 계좌에 입금하는 `deposit()`, 계좌에서 출금하는 `withdrawal()`, 한 계좌에서 다른 계좌로 이체하는 `send_money()`입니다.
- 은행 클래스의 각 메서드는 고객계좌 객체와 거래금액을 인자로 전달받아 고객계좌 객체의 거래금액만큼 잔고를 수정합니다.

```
# 계좌 클래스
class Customer_account:
    def __init__(self, name, bal):
        self.name = name
        self.balance = bal # 통장 잔고
        print(self.name, '고객님! 계좌를 개설했습니다. 잔고는', self.balance, '원')

# 은행 클래스
class Bank:
    # 입금 거래, customer에 customer_account 객체 전달
    def deposit(self, customer, amount):
        (가)
        print(customer.name, '고객님', amount, '원 입금')
        print('입금 후 잔고는', customer.balance, '원')

    # 출금 거래, customer에 customer_account 객체 전달
    def withdrawal(self, customer, amount):
        (나)
        print(customer.name, '고객님', amount, '원 출금')
        print('출금 후 잔고는', customer.balance, '원')

    # 계좌이체 거래, sender와 reciever에 customer_account 객체 전달
    def send_money(self, sender, reciever, amount):
        (다)
        print(sender.name, '고객님이', reciever.name, '고객님께', amount, '원 송금')
        print('이체 후 잔고는', sender.name, sender.balance, reciever.name, reciever.b
alance)
```

1 고객계좌의 잔고를 수정하는 코드를 (가), (나), (다)에 작성해 `Bank` 클래스를 완성하세요.

2 완성된 `customer_account`와 `bank` 클래스를 바탕으로 다음 조건을 만족하는 프로그램을 작성하세요.

|조건|
- 이름이 라이언, 잔고가 5만 원인 고객계좌 객체를 생성하고, 객체명은 `ryan`으로 하세요.
- 이름이 프로도, 잔고가 10만 원인 고객계좌 객체를 생성하고, 객체명은 `prodo`로 하세요.
- `Bank` 클래스의 객체를 생성하고 객체명은 `bank_a`로 하세요.
- 라이언이 본인 계좌에서 15,000원을 출금합니다.
- 프로도가 본인 계좌에 20,000원을 입금합니다.
- 프로도가 라이언에게 25,000원을 이체합니다.

3 은행이 거래할 때마다 수수료를 받는다고 합니다. 다음 조건을 바탕으로 `Bank` 클래스 정의를 수정하세요.

|조건|
- `Bank` 클래스에 새로운 인스턴스 속성 `commission`을 추가하고, 객체 생성 시 `commission`은 0이 되도록 수정하세요.
- 거래 시에는 고객이 입력한 금액(전달된 금액)에서 수수료를 차감한 금액이 최종 거래금액이 됩니다.
- 거래 시 발생하는 수수료는 은행 객체의 `commission`에 더합니다.
- 입금 거래와 출금 거래는 수수료가 500원입니다.
- 계좌이체 거래는 수수료가 800원입니다.

4 3번에서 완성한 코드를 바탕으로 다음 조건을 만족하는 프로그램을 작성하세요.

|조건|
- 이름이 라이언, 잔고가 5만 원인 고객계좌 객체를 생성하고, 객체명은 `ryan`으로 하세요.
- 이름이 프로도, 잔고가 10만 원인 고객계좌 객체를 생성하고, 객체명은 `prodo`로 하세요.
- `Bank` 클래스의 객체를 생성하고 이름은 `bank_a`로 하세요.
- 라이언이 본인 계좌에 10,000원을 입금합니다.
- 프로도가 라이언에게 25,000원을 이체합니다.
- 라이언이 본인 계좌에서 30,000원을 출금합니다.
- 세 건의 거래 이후 `bank_a`의 수수료 수입을 출력하세요.

정답 및 해설